

COMS3011A Lab 1

Full Chat Transcript

SDP Lab 1 planning, implementation, testing and documentation

User and assistant messages from the complete project conversation

61 messages

User**Message 1**

Hi, ive received my 1st lab for SDP, for now just read the AI policy and how we should structure our commits and documentation and stuff, and after that read the lab 1 pdf, do not start coding as yet for now just read.

Attachments: ai_policy.pdf, lab_one (1).pdf

Assistant**Message 2**

I've read both documents and won't start coding yet.

AI policy rules we must follow

AI use is **encouraged**, but everything must be transparent and verified. We remain responsible for incorrect, plagiarised, or hallucinated output. `filecite[turn0file0]`

For this project, we must record:

- The AI tool used.
- The exact model used.
- What it was used for: planning, code generation, debugging, review, writing, etc.

For any documentation we work on together, it must include an AI declaration such as:

> The preceding document was planned, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Every submitted document needs either an AI-usage declaration or an explicit statement that no AI was used. Otherwise, it will not be marked.

For commits containing AI-generated code, the commit should have a normal descriptive message followed by:

```
```text
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```
```

The README must separately declare whether the repository uses:

- AI code generation.
- AI inline editing/autocomplete.
- AI code review.

We also need to preserve transcripts covering planning, generation and debugging. The lab rubric specifically rewards **task-level AI use**, constraints being stated before generation, and at least one genuine example where you identify a bad or unsuitable suggestion and redirect me. A transcript showing the entire application generated from only the brief would score poorly or potentially zero for AI usage.

Lab 1 requirements understood

The project is a **local-first Next.js and SQLite todo application**, due **4 August 2026**. It is not deployed and has no accounts. `filecite[turn0file1]`

The required functionality is:

- Create, edit and archive tasks.
- Store title, description, due date and topic.
- Fixed statuses: ``Todo``, ``In-Progress``, and ``Complete``.
- Archived tasks remain viewable and are never deleted.
- Sort by topic, status and due date.
- Clearly flag overdue tasks without making "Overdue" a status.
- Persist all data after stopping and restarting the application.

Important implementation constraints from the rubric:

- Overdue should be **derived from the due date and status**, not stored as a database column.
- Archiving should use a flag or timestamp on the existing task.
- Data must genuinely use SQLite, not memory or JSON.
- Tests must use a temporary/throwaway database rather than your development database.
- There must be at least three meaningful tests, including archiving or overdue behaviour.
- All tests must run using one documented command.

Documentation and commit expectations

The Markdown documentation must cover:

1. **Third-Party Code** — every installed package and why it was selected.

2. **Database Design** — tables, columns and relationships matching the real schema.
3. **Running It** — exact Node version and complete install, run and test commands verified from a clean clone.

For full commit-history marks:

- At least six commits.
- Each commit should be a coherent working slice.
- Avoid vague messages such as `fix`, `update` or `wip`.
- Explain why when it is not obvious from the changes.
- Work must visibly occur over more than one session.
- Avoid generating everything first and dumping it into one or two commits.

When we begin, I'll treat this as a guided task-by-task build rather than generating the complete project at once, and I'll keep the AI attribution, transcript and commit requirements in mind throughout.

User

Message 3

Here's the constraints but I'm sure you can re-read the pdf just to double check before we get started. Don't code as yet:

- The user can create, edit and archive tasks.
- Each task carries the following information: Title, Description, Due Date, Topic.
- A task cannot be deleted, only archived, so that it remains viewable.
- The user can view their tasks as a list, sortable by topic, by status and by due date.
- Each task has one of three statuses: Todo, In-Progress, Complete. These are fixed; they are not user-customisable.
- A task that is overdue must be indicated in some way, but not as a status.
- If the application is restarted, all information persists.

Assistant

Message 4

I re-read the full Lab 1 PDF and your list matches the brief exactly. `filecite{turn0file1}`

The core constraints are:

- Tasks can be **created, edited and archived**.
- Each task must contain:
 - Title
 - Description
 - Due date
 - Topic
- Tasks must **never be deleted**. Archiving only removes them from the active list, and archived tasks must still be viewable.
- Tasks are shown as a list and must be sortable by:
 - Topic
 - Status
 - Due date
- Status is restricted to exactly:
 - `Todo`
 - `In-Progress`
 - `Complete`
- Users cannot create or customise statuses.
- Overdue is a visual condition, **not a fourth status**.
- Data must remain after a page reload and after completely stopping and restarting the application.

A few important details from the marking rubric that sit behind those constraints:

- Overdue should be calculated from the due date and task status rather than stored as an `overdue` field.
- Archiving should be represented on the original task, such as with a flag or timestamp, rather than moving the task elsewhere.
- The data must be stored in SQLite, not JSON, browser memory or an in-memory array.
- The application is local-first, runs through Node.js/npm or equivalent, has no accounts and will not be deployed.
- At least three meaningful tests are required, with at least one covering archiving or the overdue rule.
- Tests must use a disposable test database and run through one documented command.
- The repository needs accurate Markdown documentation for third-party packages, database design, and clean-clone setup instructions.
- We need at least six sensible commits, spread over more than one working session.
- AI usage must be task-level and documented through transcripts, README declarations and assisted commit attribution where applicable.

No code started yet.

User

Message 5

ok lets get started i created a repo and cloned it into VS code, what should we start with the Database perhaps

Assistant

Message 6

Yes—the **database** should be the first real feature, but we should first create a clean, runnable Next.js skeleton. That gives us a working first commit and avoids mixing setup, database code, and UI into one bulk commit.

Step 1: confirm your environment

In the VS Code terminal, from the cloned repository folder, run:

```
```bash
node -v
npm -v
```
```

Next.js currently requires Node.js 20.9 or newer, and the official starter supports TypeScript, ESLint, Tailwind and the App Router.
([nextjs.org](https://nextjs.org/docs/app/getting-started/installation?utm_source=chatgpt.com))(https://nextjs.org/docs/app/getting-started/installation?utm_source=chatgpt.com)

Send me the output before installing anything so we can record the exact Node version later in the **Running It** documentation.

Step 2: scaffold the application

Once the Node version is confirmed, we will run:

```
```bash
npx create-next-app@latest .
```
```

I recommend these selections:

```
```text
Recommended defaults? No, customize settings
TypeScript? Yes
Linter? ESLint
React Compiler? No
Tailwind CSS? Yes
Code inside src directory? Yes
App Router? Yes
Customize import alias? No
```
```

The official Next.js setup uses `create-next-app`, and TypeScript support is built into the framework.
([nextjs.org](https://nextjs.org/docs/app/getting-started/installation?utm_source=chatgpt.com))(https://nextjs.org/docs/app/getting-started/installation?utm_source=chatgpt.com)

Then we verify:

```
```bash
npm run dev
```
```

Only after the starter page works do we make the first commit:

```
```text
chore: scaffold Next.js application
```
```

That commit does **not** need an `Assisted-by` line because no project code was generated by ChatGPT; this conversation is currently planning/setup assistance.

Step 3: database design

The second task will be the database. The initial design should be one `tasks` table:

| Field | Purpose |
|---------------|---|
| `id` | Unique task identifier |
| `title` | Required task title |
| `description` | Task details |
| `due_date` | Date stored consistently |
| `topic` | Topic used for display and sorting |
| `status` | Restricted to Todo, In-Progress or Complete |

```
`archived_at`	Null while active; timestamp when archived
`created_at`	Creation timestamp
`updated_at`	Last-edit timestamp
```

Important decisions:

- No `overdue` column—it will be calculated from `due\_date` and `status`.
- No separate archive table—archiving updates `archived\_at`.
- No delete operation.
- Status will have a database constraint preventing other values.
- SQLite will use a real file for the application and a disposable database for testing.

Those choices directly target the complete database-design rubric. `filecite`turn0file1

Start by sending the results of:

```
```bash  
node -v
npm -v
```
```

User

Message 7

```
PS C:\Code\SDP> node -v  
v24.13.1  
PS C:\Code\SDP> npm -v  
11.8.0
```

Assistant

Message 8

Those versions are good:

```
```text  
Node.js: v24.13.1
npm: 11.8.0
```
```

Record **Node.js v24.13.1** because the final “Running It” documentation must state the exact version used. The current Next.js starter supports TypeScript, ESLint, Tailwind, the `src` directory and the App Router.
([nextjs.org](https://nextjs.org/docs/app/api-reference/cli/create-next-app?utm_source=chatgpt.com))

1. Scaffold the Next.js project

From:

```
```powershell  
PS C:\Code\SDP>
```

run:

```
```powershell  
npx create-next-app@latest .  
```
```

Choose:

```
```text  
Would you like to use the recommended Next.js defaults?  
No, customize settings
```

```
Would you like to use TypeScript?  
Yes
```

```
Which linter would you like to use?  
ESLint
```

```
Would you like to use React Compiler?  
No
```

```
Would you like to use Tailwind CSS?  
Yes
```

```
Would you like your code inside a src/ directory?  
Yes
```

```
Would you like to use App Router?
```

Yes

Would you like to customize the import alias?

No
```

React Compiler is not needed for this lab, and avoiding it keeps the setup simpler.

## 2. Verify the clean scaffold

After installation completes:

```powershell  
npm run dev
```

Open:

```text  
http://localhost:3000
```

Once the default Next.js page loads, stop the server with:

```text  
Ctrl + C
```

Then verify the project:

```powershell  
npm run lint
npm run build
```

Both should complete without errors.

## 3. Make the first commit

Check what was created:

```powershell  
git status
```

Then:

```powershell  
git add .
git commit -m "chore: scaffold Next.js application"
git push
```

This is a sensible first coherent commit and begins the required commit history before database functionality is added. The rubric requires at least six coherent commits rather than one or two bulk dumps. [filecite]turn0file1[

This commit does **not** need an `Assisted-by` footer because it contains the standard `create-next-app` scaffold rather than AI-generated project code. This conversation should still be retained as part of the planning transcript. After the scaffold runs successfully, the second commit will be the SQLite schema and database connection.

**User**

Message 9

Could not create a project called "SDP" because of npm naming restrictions:

\* name can no longer contain capital letters  
npm notice  
npm notice New minor version of npm available! 11.8.0 -> 11.19.0  
npm notice Changelog: <https://github.com/npm/cli/releases/tag/v11.19.0>  
npm notice To update run: npm install -g npm@11.19.0  
npm notice  
PS C:\Code\SDP>

**Assistant**

Message 10

That happened because `create-next-app` uses the **current folder name** as the npm package name. Your folder is named `SDP`, but npm package names cannot contain uppercase letters. ([docs.npmjs.com](https://docs.npmjs.com/package-name-guidelines?utm\_source=chatgpt.com))

Rename the local folder to lowercase. Because Windows can be awkward when changing only the capitalisation, use a temporary name:

```
```powershell
cd ..
```

```
Rename-Item SDP sdp-temp
Rename-Item sdp-temp sdp
```

```
cd sdp
code .
```
```

Then, in the newly opened VS Code terminal:

```
```powershell
npx create-next-app@latest .
```
```

Renaming the local folder will **\*\*not affect the GitHub repository or its remote connection\*\*** because the `.git` directory moves with it.

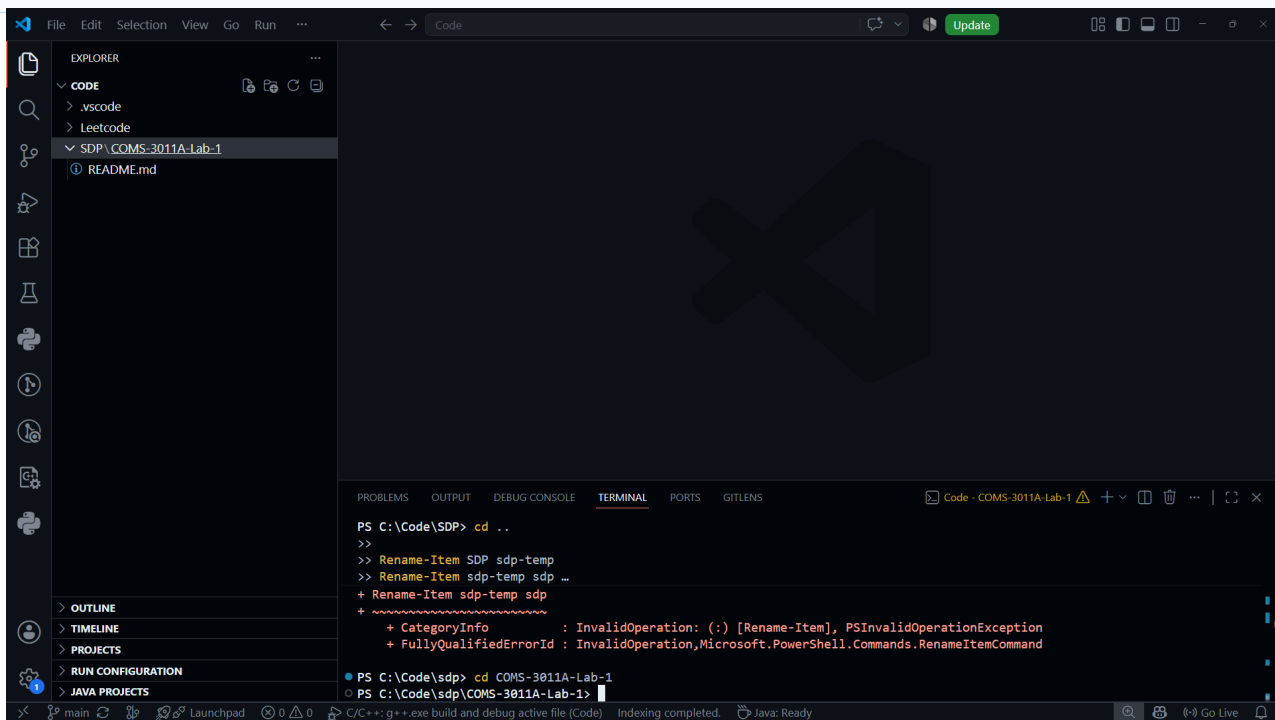
The npm update notice is unrelated. Leave npm on `11.8.0` for now so we do not introduce unnecessary environment changes during the lab setup.

## User

Message 11

[Screenshot uploaded]

i forgot to change to the correct directory cause i made a folder for the lab mb we now using this do u wanna revert the sdp back to SDP and then continue



## Assistant

Message 12

Yes, revert the parent folder to ``SDP``, but the **\*\*actual repository folder also needs to be lowercase\*\*** because ``create-next-app`` uses the current folder name for the npm package name.

Your current path is:

```
```text
C:\Code\sdp\COMS-3011A-Lab-1
```
```

``COMS-3011A-Lab-1`` will still trigger the same uppercase-name error.

Close VS Code first so Windows is not holding the folders open. Then open a separate PowerShell window and run:

```
```powershell
```

```
cd C:\Code
```

```
Rename-Item sdp sdp-temp  
Rename-Item sdp-temp SDP
```

```
cd .\SDP
```

```
Rename-Item COMS-3011A-Lab-1 lab1-temp  
Rename-Item lab1-temp coms-3011a-lab-1
```

```
cd .\coms-3011a-lab-1  
code .  
```
```

Your resulting local path will be:

```
```text  
C:\Code\SDP\coms-3011a-lab-1  
```
```

The GitHub repository name can remain `COMS-3011A-Lab-1`; renaming the local folder does not affect the remote repository or Git history.

Also, the `...` shown after your earlier command should not be typed. PowerShell interpreted that as part of the command, which contributed to the error.

Once the new VS Code window opens, confirm the terminal says:

```
```text  
PS C:\Code\SDP\coms-3011a-lab-1>  
```
```

Then run:

```
```powershell  
npx create-next-app@latest .  
```
```

## User

Message 13

```
PS C:\Code\SDP\coms-3011a-lab-1> npx create-next-app@latest .
The directory coms-3011a-lab-1 contains files that could conflict:
```

```
 README.md
```

Either try using a new directory name, or remove the files listed above.

```
PS C:\Code\SDP\coms-3011a-lab-1>
```

## Assistant

Message 14

That is just the default `README.md` created with the GitHub repository. `create-next-app` refuses to overwrite it.

Keep a backup, scaffold the project, then we can replace the generated README with the proper lab documentation later.

Run:

```
```powershell  
Move-Item .\README.md ..\README-lab1-original.md  
  
npx create-next-app@latest .  
```
```

Choose:

```
```text  
Recommended defaults? No, customize settings  
TypeScript? Yes  
Linter? ESLint  
React Compiler? No  
Tailwind CSS? Yes  
Code inside src directory? Yes  
App Router? Yes  
Customize import alias? No  
```
```

The backup will sit here:

```
```text
C:\Code\SDP\README-lab1-original.md
```
```

After the setup finishes, verify:

```
```powershell
npm run dev
```
```

Then open:

```
```text
http://localhost:3000
```
```

Do not commit yet—we'll first confirm the application runs, lint passes, and the generated files look correct.

**User**

Message 15

Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code

**Assistant**

Message 16

Choose **\*\*Yes\*\***.

`AGENTS.md` does not activate an AI agent or generate code. It gives any coding agent working in the repository instructions to consult the version-matched Next.js documentation bundled with the installed `next` package, reducing outdated suggestions.  
([nextjs.org](https://nextjs.org/docs/app/guides/ai-agents?utm\_source=chatgpt.com))

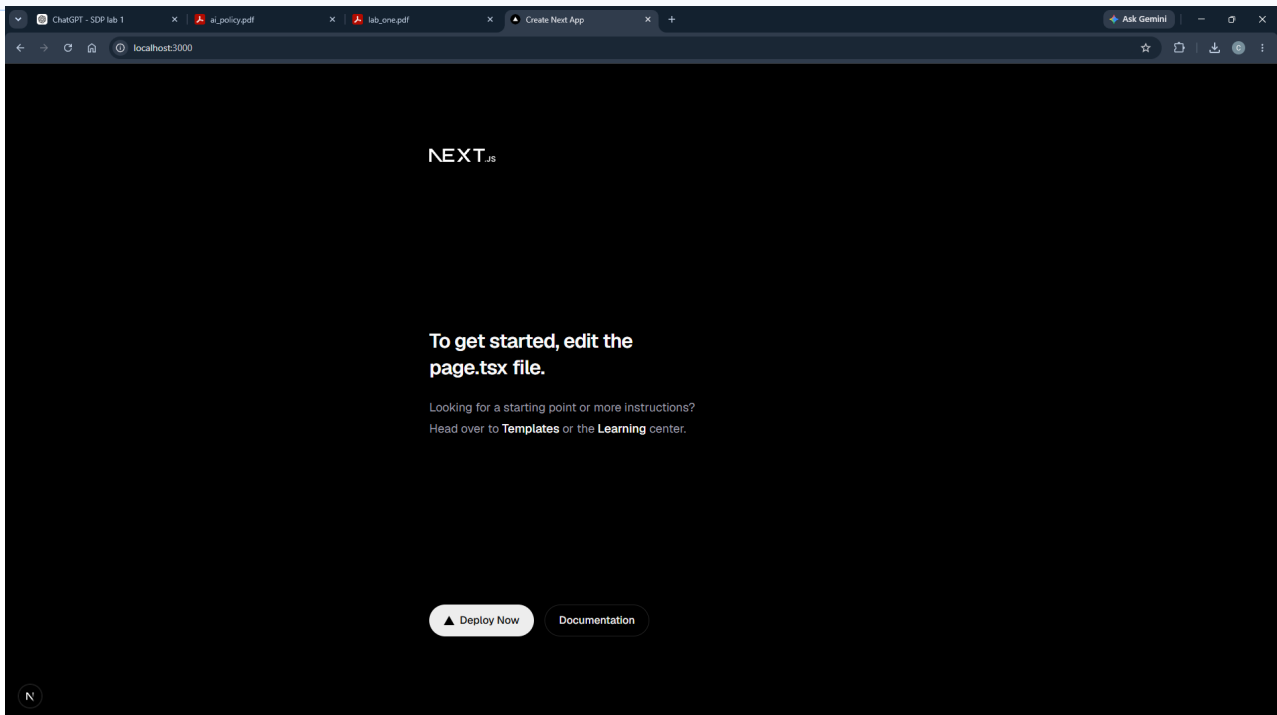
It is safe for this lab and can remain as part of the scaffold. It does **\*\*not\*\*** by itself count as AI code generation or require an `Assisted-by` commit footer.

**User**

Message 17

[Screenshot uploaded]

ok Ws lets continue



**Assistant**

Message 18

W ☐ The default page loading confirms the scaffold starts correctly.

## Finish and commit the scaffold first

Stop the development server with:

```
```powershell
Ctrl + C
```
```

Then run:

```
```powershell
npm run lint
npm run build
```
```

Both should complete successfully. After that:

```
```powershell
git status
git add .
git commit -m "chore: scaffold Next.js application"
git push
```
```

This first commit does **not** need an `Assisted-by`` footer because it contains the standard `create-next-app`` scaffold rather than AI-generated project code. We should still retain this conversation as part of the planning transcript.

## What we'll do next

The second coherent slice will be the database foundation:

1. Install the SQLite package.
2. Define the `tasks`` schema in a visible SQL file.
3. Create the database connection and initialization logic.
4. Store the development database in an ignored `data`` directory.
5. Verify that the table and constraints are created correctly.
6. Commit it separately as the database-schema commit.

I recommend using `better-sqlite3`` directly rather than adding an ORM. It keeps the schema obvious, makes constraints easy to inspect, and suits a small local-first application. Current releases support maintained Node versions including Node 24.  
([github.com](https://github.com/WiseLibs/better-sqlite3/releases?utm_source=chatgpt.com))([https://github.com/WiseLibs/better-sqlite3/releases?utm\\_source=chatgpt.com](https://github.com/WiseLibs/better-sqlite3/releases?utm_source=chatgpt.com)))

The initial table will follow the rubric:

```
```text
tasks
-----
id
title
description
due_date
topic
status
archived_at
created_at
updated_at
```
```

There will be no stored `overdue`` field, no archive table and no delete operation. The rubric expects overdue to be derived and archiving to remain on the original task record.  
`filecite[turn0file1]`

Run the lint, build and commit commands first, then send the terminal output.

**User**

Message 19

ok lets continue

**Assistant**

Message 20

W, now we'll do the **database foundation as the second coherent commit**—no UI and no CRUD yet.

Before continuing, confirm the scaffold commit exists:

```
```powershell
git log -1 --oneline
```
```

You should see something like:

```
```text
chore: scaffold Next.js application
```
```

### ## Database task constraints

For this slice:

- Use SQLite.
- Keep a visible SQL schema.
- Use one `tasks` table.
- Restrict status to exactly `Todo`, `In-Progress`, or `Complete`.
- Represent archiving using `archived\_at`.
- Do not create a delete operation.
- Do not store an `overdue` column; overdue will later be derived from due date and status.
- Make the database connection reusable and testable.

These decisions directly match the complete database-design requirements in the lab rubric.  
[\[filecite\]turn0file1](#)

We'll use `better-sqlite3`. It provides a simple synchronous SQLite API, supports currently maintained Node.js versions, and Next.js automatically handles it as an external server package. ([github.com](https://github.com/WiseLibs/better-sqlite3))

### ## 1. Install the database dependency

Run:

```
```powershell
npm install better-sqlite3
npm install --save-dev @types/better-sqlite3
```
```

### ## 2. Create the schema

Create this folder and file:

```
```text
database/
└─ schema.sql
```
```

Put this in `database/schema.sql`:

```
```sql
CREATE TABLE IF NOT EXISTS tasks (
  id INTEGER PRIMARY KEY AUTOINCREMENT,

  title TEXT NOT NULL
    CHECK (length(trim(title)) > 0),

  description TEXT NOT NULL DEFAULT "",

  due_date TEXT NOT NULL
    CHECK (
      due_date GLOB
      '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]'
    ),

  topic TEXT NOT NULL
    CHECK (length(trim(topic)) > 0),

  status TEXT NOT NULL DEFAULT 'Todo'
    CHECK (status IN ('Todo', 'In-Progress', 'Complete')),

  archived_at TEXT DEFAULT NULL,

  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,

  updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
) STRICT;

CREATE INDEX IF NOT EXISTS idx_tasks_topic
ON tasks(topic);

CREATE INDEX IF NOT EXISTS idx_tasks_status
ON tasks(status);
```
```

```
CREATE INDEX IF NOT EXISTS idx_tasks_due_date
 ON tasks(due_date);

CREATE INDEX IF NOT EXISTS idx_tasks_archived_at
 ON tasks(archived_at);
````
```

Important design meanings:

- `archived\_at IS NULL` means active.
- A timestamp in `archived\_at` means archived.
- `due\_date` uses the sortable `YYYY-MM-DD` format.
- There is deliberately no `overdue` column.
- The database itself rejects invalid statuses.
- `STRICT` prevents values of unsuitable SQLite storage types.

3. Create the reusable database factory

Create:

```
````text
src/lib/db/database.ts
````
```

Add:

```
````typescript
import Database from "better-sqlite3";
import { mkdirSync, readFileSync } from "node:fs";
import path from "node:path";

const schemaPath = path.join(
 process.cwd(),
 "database",
 "schema.sql",
);

export function openDatabase(databasePath: string) {
 if (databasePath !== ":memory:") {
 mkdirSync(path.dirname(databasePath), {
 recursive: true,
 });
 }

 const database = new Database(databasePath, {
 timeout: 5000,
 });

 try {
 database.pragma("foreign_keys = ON");

 if (databasePath !== ":memory:") {
 database.pragma("journal_mode = WAL");
 }

 const schema = readFileSync(schemaPath, "utf8");
 database.exec(schema);

 return database;
 } catch (error) {
 database.close();
 throw error;
 }
}
````
```

The factory accepts a path rather than hard-coding one. Later:

- The real application can use `data/tasks.db`.
- Tests can use `:memory:` or a temporary file.
- Tests therefore won't depend on your personal development database.

4. Create the application connection

Create:

```
````text
src/lib/db/index.ts
````
```

Add:

```
```typescript
import "server-only";

import path from "node:path";

import { openDatabase } from "../database";

const databasePath =
 process.env.DATABASE_PATH ??
 path.join(process.cwd(), "data", "tasks.db");

const globalForDatabase = globalThis as typeof globalThis & {
 taskDatabase?: ReturnType<typeof openDatabase>;
};

export const db =
 globalForDatabase.taskDatabase ??
 openDatabase(databasePath);

if (process.env.NODE_ENV !== "production") {
 globalForDatabase.taskDatabase = db;
}
```
```

The global reference prevents development hot reloads from repeatedly opening unnecessary database connections.

The `server-only` marker prevents this native database module from accidentally being imported into a browser component. Database queries belong on the server side of the Next.js application.

([nextjs.org](https://nextjs.org/docs/app/getting-started/server-and-client-components?utm_source=chatgpt.com))

5. Ignore the generated database files

At the bottom of `.gitignore`, add:

```
```gitignore
Local SQLite database files
/data/
```
```

The database must persist locally, but your personal database contents should not be pushed to GitHub. The schema remains committed, so a clean clone can create its own database.

6. Add the required AI declaration

Because this is now AI-assisted code generation, add this section to the bottom of `README.md`:

```
```markdown
AI Usage
```

This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

This repository does not use AI in-line editing tools.

This repository does not use AI code review.

We can change the code-review line later once I formally review implementation written or modified by you. The AI policy requires repository-level attribution once AI code generation is used. [filecite]turn0file0

### ## 7. Verify the schema

Run this as one PowerShell command:

```
```powershell
node -e "const Database=require('better-sqlite3'); const fs=require('node:fs'); const db=new Database(':memory:'); db.exec(fs.readFileSync('database/schema.sql','utf8')); console.table(db.prepare('PRAGMA table_info(tasks)').all()); db.close();"
```
```

You should see these columns:

```
```text
```

```
id
title
description
due_date
topic
status
archived_at
created_at
updated_at
```
```

Then run:

```
```powershell
npm run lint
npm run build
git status
```
```

## 8. Commit this database slice

Once everything passes:

```
```powershell
git add .
git commit `
  -m "feat: add SQLite database foundation" `
  -m "Define the constrained task schema and reusable server-only database connection." `
  -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```
```

```
git push
```
```

This commit contains only the schema and database infrastructure, leaving CRUD operations for the next coherent commit.

User

Message 21

ok Ws stop coding lets talk for a bit can i send u anything so that u can verify what we done so far is right

Assistant

Message 22

Yes. The best option is to upload a **ZIP of the repository** so I can verify the structure and actual code together.

Before zipping it, exclude:

```
```text
node_modules/
.next/
data/
*.db
*.db-wal
*.db-shm
```
```

Do **not** include `.env` files or anything containing secrets.

Also paste the output of:

```
```powershell
git log --oneline --decorate -5
git status
npm run lint
npm run build
```
```

I'll check:

- The Next.js scaffold and folder structure.
- `package.json` and installed dependencies.
- `database/schema.sql`.
- `src/lib/db/database.ts`.
- `src/lib/db/index.ts`.
- `.gitignore`.
- README AI declarations.
- Whether the schema matches the rubric.
- Whether the commits and attribution follow the AI policy.

- Any build, TypeScript or lint problems.

A GitHub repository link also works, but uploading the ZIP here is usually easier for a full review.

User

Message 23

```
a60ae21 (HEAD -> main, origin/main, origin/HEAD) feat: add SQLite database foundation
eeb4da9 chore: scaffold Next.js application
ccd6640 Initial commit
On branch main
Your branch is up to date with 'origin/main'.
```

nothing to commit, working tree clean

```
> coms-3011a-lab-1@0.1.0 lint
> eslint
```

```
> coms-3011a-lab-1@0.1.0 build
> next build
```

▲ Next.js 16.2.12 (Turbopack)

```
  Creating an optimized production build ...
  ✓ Compiled successfully in 2.9s
  ✓ Finished TypeScript in 2.6s
  ✓ Collecting page data using 5 workers in 891ms
  ✓ Generating static pages using 5 workers (4/4) in 1009ms
  ✓ Finalizing page optimization in 15ms
```

Route (app)

```
├─ ○ /
└─ ○ /_not-found
```

○ (Static) prerendered as static content

Assistant

Message 24

From that output, the project foundation is in a good state.

What is already confirmed:

- The repository is synced with GitHub.
- The working tree is clean.
- The scaffold and database work were separated into distinct commits.
- ESLint passes.
- TypeScript passes.
- The production build succeeds.
- Next.js recognises the application correctly.
- There are currently three commits, so you are halfway to the rubric's minimum of six coherent commits. The final history must also show work spread over more than one session.

📄filecite🔄turn0file1📄

The current history also looks sensible:

```
```text
ccd6640 Initial commit
eeb4da9 chore: scaffold Next.js application
a60ae21 feat: add SQLite database foundation
```
```

One thing this output cannot verify is whether the database commit contains the required AI attribution footer. To check it, run:

```
```powershell
git show -s --format=full a60ae21
```
```

Near the bottom, it should contain:

```
```text
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```
```

The one-line Git log hides commit-body text, so the footer may already be there even though it is not displayed. The AI policy requires that attribution for commits containing AI-generated code. 📄filecite🔄turn0file0📄

I also cannot yet verify the actual schema constraints, database connection code, `.gitignore`, dependency versions, or README declaration from terminal output alone. Uploading a ZIP of the repository—without `node_modules`, `.next`, `data`, database files, or secrets—would let me inspect everything we have done so far without changing or coding anything.

User

Message 25

its ok theres nothing to hide heres the repo link check it, we barely done anything :
<https://github.com/CalebP987/COMS-3011A-Lab-1>

Assistant

Message 26

Yeah, I checked the actual `main` branch. For where we are now, everything is correct and clean.

What is right

- The repository has the expected Next.js structure plus the separate `database/schema.sql` and `src/lib/db` files. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1))
- `better-sqlite3` and its TypeScript definitions are recorded properly in `package.json`. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1/raw/refs/heads/main/package.json))
- The schema uses one persistent `tasks` table with:
 - all four required task fields;
 - exactly the three allowed statuses;
 - `archived_at` rather than deleting or moving tasks;
 - no stored overdue field;
 - useful indexes for the required sorting and archive filtering.
- The database factory creates the database directory, applies the schema, enables foreign keys and uses WAL mode for the real database. The separate application connection points to `data/tasks.db` and is reusable during development. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1/blob/main/database/schema.sql))
- The database factory creates the database directory, applies the schema, enables foreign keys and uses WAL mode for the real database. The separate application connection points to `data/tasks.db` and is reusable during development. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1/commit/a60ae21))
- `/data/` is ignored, so the schema is committed without uploading your personal SQLite contents. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1/blob/main/.gitignore))
- The database commit is a coherent slice, and the AI attribution is present in its commit description. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1/commit/a60ae21))
- The README correctly records ChatGPT code generation. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1))

Together with your successful lint and production build, there is nothing broken that we need to undo.

One correction from my earlier instructions

The README currently says:

> This repository does not use AI code review.

That was accurate before, but it is no longer accurate because you have now explicitly asked me to inspect and verify the repository. The AI policy says AI code review must be declared in the README. [filecite]turn0file0

When we resume, that should become:

```
```text
This repository makes use of AI code-review using the following tools: ChatGPT-Web[GPT-5.6
Thinking].
```
```

This is actually useful for the transcript because it shows us noticing that an earlier statement became unsuitable and correcting it rather than blindly leaving it.

Things to remember later, not current errors

The README is still mostly the default Next.js README and talks about Vercel deployment. Before submission, we must replace it with the actual `**Third-Party Code**`, `**Database Design**`, and `**Running It**` documentation required by the lab. ([github.com](https://github.com/CalebP987/COMS-3011A-Lab-1/blob/main/README.md)) [filecite]turn0file1

Also:

- The current due-date constraint confirms the `YYYY-MM-DD` shape, but it could still accept something impossible such as `2026-99-99`. We should perform proper date validation when we create the task service.
- `updated_at` currently has an initial default only. Our edit query must explicitly update it later.

- The database file will only be created once application code actually imports and uses the database connection.

So overall: ****database foundation is approved; no rewrite needed****. We caught one documentation declaration that needs updating because this review itself changed the AI-usage situation.

User

Message 27

ok i updated the code review, so the DB is created right? if so thats a good place to stop for this session, next session we can build backend and frontend and stuff and 3rd and last session we can connect APIs and test and stuff if thats fine or should we do more now

Assistant

Message 28

Yes—the ****database foundation is created correctly****, with one distinction:

- `database/schema.sql` defines the real `tasks` table and its constraints.
([\[raw.githubusercontent.com\]\(https://raw.githubusercontent.com/CalebP987/COMS-3011A-Lab-1/main/database/schema.sql\)](https://raw.githubusercontent.com/CalebP987/COMS-3011A-Lab-1/main/database/schema.sql))
- `openDatabase()` creates the database directory, opens SQLite and applies that schema.
([\[raw.githubusercontent.com\]\(https://raw.githubusercontent.com/CalebP987/COMS-3011A-Lab-1/main/src/lib/db/database.ts\)](https://raw.githubusercontent.com/CalebP987/COMS-3011A-Lab-1/main/src/lib/db/database.ts))
- The application database is configured to live at `data/tasks.db`.
([\[raw.githubusercontent.com\]\(https://raw.githubusercontent.com/CalebP987/COMS-3011A-Lab-1/main/src/lib/db/index.ts\)](https://raw.githubusercontent.com/CalebP987/COMS-3011A-Lab-1/main/src/lib/db/index.ts))

The physical `data/tasks.db` file is only created when application code imports and uses the database connection. Since the homepage does not use the database yet, that file may not exist currently. That is completely normal—the schema and initialization system are ready, and the file will appear automatically once we implement the task backend.

This is a good stopping point

Stopping now actually helps the commit-history rubric because the work must visibly occur over more than one session. We have completed one coherent slice:

1. Project scaffold.
2. SQLite schema and connection foundation.
3. Successful lint and production build.

Before closing VS Code, commit and push your README code-review correction. GitHub currently still displays the old “does not use AI code review” line, so the change appears to be local or not pushed yet.
([\[github.com\]\(https://github.com/CalebP987/COMS-3011A-Lab-1/blob/main/README.md\)](https://github.com/CalebP987/COMS-3011A-Lab-1/blob/main/README.md))

```
```powershell
git status
git add README.md
git commit -m "docs: declare AI code review usage"
git push
```
```

Revised plan for the remaining sessions

Your three-session plan is achievable, but we should ****not build the backend and frontend separately and only connect them at the end****.

Session 2

Build a working vertical slice:

- Task TypeScript types and validation.
- Database functions for creating, reading, editing and archiving tasks.
- Server actions or route handlers.
- Basic integrated create-and-list interface.
- Start the behavioural tests against a throwaway database.

This means we finish the session with tasks genuinely entering SQLite and appearing on screen.

Session 3

Complete and verify everything:

- Edit tasks.
- Change the fixed status.
- Archive tasks and view archived tasks.
- Sort by topic, status and due date.
- Derive and display overdue state.

- Finish at least three deterministic tests, including archiving or overdue.
- Replace the default README with the required third-party code, database design and clean-clone instructions.
- Perform the complete seven-step marking walkthrough from a clean clone.

The lab gives major marks to testing and documentation, so we should develop tests alongside the backend rather than leaving every test for the final few hours. `filecite`turn0file1

So yes: **push the README correction and stop for tonight. The database session is complete.**

User

Message 29

nah thats not worth of a commit it will be there next time, ill see u then

Assistant

Message 30

Fair 🤖 leave it for the next proper commit. Good stopping point for session 1—the scaffold and database foundation are done, clean, and pushed.

See you next session for the task backend and first working UI slice

User

Message 31

ok im back its another session, lets try to finish the frontend and backend today and in the next and final session we will do APIs if needed, testing and documentation, we can begin

Assistant

Message 32

W, session 2 begins

We'll target **three coherent commits today**:

1. Task domain, validation and SQLite repository.
2. Create/list functionality with the first working interface.
3. Edit, archive, sorting and archived-task view.

That will leave testing, final documentation and clean-clone verification for session 3.

Architecture decision

We **do not need REST API routes** for this lab. The application is a single local Next.js application, so we can:

- Read SQLite directly from Server Components.
- Use Server Actions for create, edit and archive operations.
- Keep all SQL inside a testable repository layer.

Next.js specifically describes Server Actions as the mechanism for frontend-triggered data mutations, while calling Route Handlers from Server Components adds an unnecessary internal HTTP round trip.

(`nextjs.org`)(https://nextjs.org/docs/app/guides/forms?utm_source=chatgpt.com)

The lab requires behaviour, not a REST API. `filecite`turn0file1

Task 1 constraints for the AI transcript

For this backend slice:

- No delete function.
- No stored `overdue` column.
- Archived tasks remain in the same table.
- Status remains restricted to the three fixed values.
- No extra validation package.
- Repository functions receive a database connection so tests can later use a disposable database.
- Overdue calculations accept a date parameter so tests can be deterministic.

Your existing repository already has the SQLite connection and schema needed for this layer. (`github.com`)(<https://github.com/CalebP987/COMS-3011A-Lab-1/blob/main/database/schema.sql>)

First check:

```
```powershell
git status
```
```

Keep your pending README code-review correction; we'll include it in this meaningful commit.

1. Create the task types

Create:

```
```text
src/lib/tasks/types.ts
```
```

```
```typescript
export const TASK_STATUSES = [
 "Todo",
 "In-Progress",
 "Complete",
] as const;

export type TaskStatus = (typeof TASK_STATUSES)[number];

export const TASK_SORT_OPTIONS = [
 "topic",
 "status",
 "dueDate",
] as const;

export type TaskSortOption =
 (typeof TASK_SORT_OPTIONS)[number];

export interface TaskInput {
 title: string;
 description: string;
 dueDate: string;
 topic: string;
 status: TaskStatus;
}

export interface Task extends TaskInput {
 id: number;
 archivedAt: string | null;
 createdAt: string;
 updatedAt: string;
 isOverdue: boolean;
}

export interface ListTaskOptions {
 archived?: boolean;
 sortBy?: TaskSortOption;
}
```
```

2. Add validation and overdue logic

Create:

```
```text
src/lib/tasks/validation.ts
```
```

```
```typescript
import {
 TASK_SORT_OPTIONS,
 TASK_STATUSES,
 type TaskInput,
 type TaskSortOption,
 type TaskStatus,
} from "./types";

export class TaskValidationError extends Error {
 constructor(message: string) {
 super(message);
 this.name = "TaskValidationError";
 }
}

export function isTaskStatus(
 value: string,
): value is TaskStatus {
 return TASK_STATUSES.some((status) => status === value);
}
```

```

}

export function isTaskSortOption(
 value: string,
): value is TaskSortOption {
 return TASK_SORT_OPTIONS.some((option) => option === value);
}

export function isValidDateString(value: string): boolean {
 if (!/^\d{4}-\d{2}-\d{2}$/.test(value)) {
 return false;
 }

 const [year, month, day] = value
 .split("-")
 .map(Number);

 const date = new Date(Date.UTC(year, month - 1, day));

 return (
 date.getUTCFullYear() === year &&
 date.getUTCMonth() === month - 1 &&
 date.getUTCDate() === day
);
}

export function validateTaskInput(
 input: TaskInput,
): TaskInput {
 const title = input.title.trim();
 const description = input.description.trim();
 const topic = input.topic.trim();
 const dueDate = input.dueDate.trim();

 if (title.length === 0) {
 throw new TaskValidationError(
 "A task title is required.",
);
 }

 if (topic.length === 0) {
 throw new TaskValidationError(
 "A task topic is required.",
);
 }

 if (!isValidDateString(dueDate)) {
 throw new TaskValidationError(
 "The due date must be a valid date.",
);
 }

 if (!isTaskStatus(input.status)) {
 throw new TaskValidationError(
 "The task status is invalid.",
);
 }

 return {
 title,
 description,
 topic,
 dueDate,
 status: input.status,
 };
}

function localDateString(date: Date): string {
 const year = date.getFullYear();
 const month = String(date.getMonth() + 1).padStart(
 2,
 "0",
);
 const day = String(date.getDate()).padStart(2, "0");

 return `${year}-${month}-${day}`;
}

export function isTaskOverdue(
 dueDate: string,

```

```

status: TaskStatus,
currentDate = new Date(),
): boolean {
 return (
 status !== "Complete" &&
 dueDate < localDateString(currentDate)
);
}
...

```

Important decision: a completed task is not displayed as overdue, even when its due date is in the past. That is why overdue is derived from both the due date and status, matching the rubric.

---

### # 3. Add the SQLite repository

Create:

```

```text
src/lib/tasks/repository.ts
```

```typescript
import type Database from "better-sqlite3";

import {
  type ListTaskOptions,
  type Task,
  type TaskInput,
  type TaskSortOption,
  type TaskStatus,
} from "../types";
import {
  isTaskOverdue,
  validateTaskInput,
} from "../validation";

interface TaskRow {
  id: number;
  title: string;
  description: string;
  due_date: string;
  topic: string;
  status: TaskStatus;
  archived_at: string | null;
  created_at: string;
  updated_at: string;
}

export class TaskNotFoundError extends Error {
  constructor(id: number) {
    super(`Task ${id} was not found.`);
    this.name = "TaskNotFoundError";
  }
}

function mapTaskRow(
  row: TaskRow,
  currentDate = new Date(),
): Task {
  return {
    id: row.id,
    title: row.title,
    description: row.description,
    dueDate: row.due_date,
    topic: row.topic,
    status: row.status,
    archivedAt: row.archived_at,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
    isOverdue: isTaskOverdue(
      row.due_date,
      row.status,
      currentDate,
    ),
  };
}

```

```

function orderByClause(
  sortBy: TaskSortOption,
): string {
  switch (sortBy) {
    case "topic":
      return `
        topic COLLATE NOCASE ASC,
        due_date ASC,
        id ASC
      `;

    case "status":
      return `
        CASE status
          WHEN 'Todo' THEN 1
          WHEN 'In-Progress' THEN 2
          WHEN 'Complete' THEN 3
        END ASC,
        due_date ASC,
        id ASC
      `;

    case "dueDate":
      return `
        due_date ASC,
        id ASC
      `;
  }
}

export function listTasks(
  database: Database.Database,
  options: ListTaskOptions = {},
  currentDate = new Date(),
): Task[] {
  const archived = options.archived ?? false;
  const sortBy = options.sortBy ?? "dueDate";

  const archiveCondition = archived
    ? "archived_at IS NOT NULL"
    : "archived_at IS NULL";

  const statement = database.prepare(`
    SELECT
      id,
      title,
      description,
      due_date,
      topic,
      status,
      archived_at,
      created_at,
      updated_at
    FROM tasks
    WHERE ${archiveCondition}
    ORDER BY ${orderByClause(sortBy)}
  `);

  return (statement.all() as TaskRow[]).map((row) =>
    mapTaskRow(row, currentDate),
  );
}

export function findTaskById(
  database: Database.Database,
  id: number,
  currentDate = new Date(),
): Task | null {
  const statement = database.prepare(`
    SELECT
      id,
      title,
      description,
      due_date,
      topic,
      status,
      archived_at,
      created_at,
      updated_at
    FROM tasks
  `);

```

```

    WHERE id = ?
`);

const row = statement.get(id) as TaskRow | undefined;

return row ? mapTaskRow(row, currentDate) : null;
}

export function createTask(
  database: Database.Database,
  input: TaskInput,
): Task {
  const task = validateTaskInput(input);

  const statement = database.prepare(`
    INSERT INTO tasks (
      title,
      description,
      due_date,
      topic,
      status
    )
    VALUES (?, ?, ?, ?, ?)
  `);

  const result = statement.run(
    task.title,
    task.description,
    task.dueDate,
    task.topic,
    task.status,
  );

  const createdTask = findTaskById(
    database,
    Number(result.lastInsertRowid),
  );

  if (!createdTask) {
    throw new Error("The created task could not be loaded.");
  }

  return createdTask;
}

export function updateTask(
  database: Database.Database,
  id: number,
  input: TaskInput,
): Task {
  const task = validateTaskInput(input);

  const statement = database.prepare(`
    UPDATE tasks
    SET
      title = ?,
      description = ?,
      due_date = ?,
      topic = ?,
      status = ?,
      updated_at = CURRENT_TIMESTAMP
    WHERE id = ?
    AND archived_at IS NULL
  `);

  const result = statement.run(
    task.title,
    task.description,
    task.dueDate,
    task.topic,
    task.status,
    id,
  );

  if (result.changes === 0) {
    throw new TaskNotFoundError(id);
  }

  const updatedTask = findTaskById(database, id);

```

```

    if (!updatedTask) {
      throw new TaskNotFoundError(id);
    }

    return updatedTask;
  }

export function archiveTask(
  database: Database.Database,
  id: number,
): Task {
  const statement = database.prepare(`
    UPDATE tasks
    SET
      archived_at = CURRENT_TIMESTAMP,
      updated_at = CURRENT_TIMESTAMP
    WHERE id = ?
      AND archived_at IS NULL
  `);

  const result = statement.run(id);

  if (result.changes === 0) {
    throw new TaskNotFoundError(id);
  }

  const archivedTask = findTaskById(database, id);

  if (!archivedTask) {
    throw new TaskNotFoundError(id);
  }

  return archivedTask;
}
...

```

This layer deliberately contains **no delete function**.

4. Add the application-facing service

Create:

```

```text
src/lib/tasks/index.ts
```

```typescript
import { db } from "@lib/db";

import {
 archiveTask as archiveTaskInDatabase,
 createTask as createTaskInDatabase,
 findTaskById as findTaskByIdInDatabase,
 listTasks as listTasksInDatabase,
 updateTask as updateTaskInDatabase,
} from "../repository";
import type {
 ListTaskOptions,
 TaskInput,
} from "../types";

export function getTasks(options?: ListTaskOptions) {
 return listTasksInDatabase(db, options);
}

export function getTask(id: number) {
 return findTaskByIdInDatabase(db, id);
}

export function addTask(input: TaskInput) {
 return createTaskInDatabase(db, input);
}

export function editTask(id: number, input: TaskInput) {
 return updateTaskInDatabase(db, id, input);
}

export function archiveTask(id: number) {

```

```
 return archiveTaskInDatabase(db, id);
 }
}
```

```
export * from "./types";
export {
 isTaskOverdue,
 isTaskSortOption,
 isTaskStatus,
 isValidDateString,
 TaskValidationError,
} from "./validation";
````
```

5. Verify this slice

Run:

```
``powershell
npm run lint
npm run build
git status
``
```

The build may now create:

```
``text
data/tasks.db
``
```

That is expected and should remain ignored by Git.

Then inspect the changes:

```
``powershell
git diff
``
```

Do ****not** commit yet**. Paste the lint/build output and any errors first, and we'll review this backend slice before making commit 4.

User

Message 33

```
PS C:\Code\SDP\coms-3011a-lab-1> npm run lint
>> npm run build
>> git status
```

```
> coms-3011a-lab-1@0.1.0 lint
> eslint
```

```
> coms-3011a-lab-1@0.1.0 build
> next build
```

▲ Next.js 16.2.12 (Turbopack)

```
  Creating an optimized production build ...
✓ Compiled successfully in 3.3s
✓ Finished TypeScript in 2.7s
✓ Collecting page data using 5 workers in 813ms
✓ Generating static pages using 5 workers (4/4) in 993ms
✓ Finalizing page optimization in 13ms
```

Route (app)

```
├ ○ /
└ ○ /_not-found
```

○ (Static) prerendered as static content

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

```
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    modified:   README.md
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
```

src/lib/tasks/

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Code\SDP\coms-3011a-lab-1> git diff
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git
touches it
diff --git a/README.md b/README.md
index a85ba8f..2527530 100644
--- a/README.md
+++ b/README.md
@@ -41,4 +41,4 @@ This repository makes use of AI code generation using the following tools:
ChatG
```

This repository does not use AI in-line editing tools.

```
-This repository does not use AI code review.
:...skipping...
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git
touches it
diff --git a/README.md b/README.md
index a85ba8f..2527530 100644
--- a/README.md
+++ b/README.md
@@ -41,4 +41,4 @@ This repository makes use of AI code generation using the following tools:
ChatG
```

This repository does not use AI in-line editing tools.

```
-This repository does not use AI code review.
\ No newline at end of file
+This repository makes use of AI code-review using the following tools: ChatGPT-Web[GPT-5.6
Thinking].
\ No newline at end of file
```

Assistant

Message 34

Everything looks good so far.

Two small notes:

- `git diff` only shows tracked-file changes, so it does **not** display the new untracked `src/lib/tasks/` files. That is expected.
- The `LF will be replaced by CRLF` warning is normal on Windows and not an error.

Your results confirm:

- ESLint passes.
- TypeScript compilation passes.
- The production build succeeds.
- The backend files are recognised without type errors.
- The README AI-review declaration is ready to be included.

This verifies the backend structurally, although the actual create/edit/archive behaviour will only be fully verified once we add the interface and tests.

Before committing, open `README.md`, go to the very end of the file, and press Enter once so it has a final newline. Then commit the whole backend slice:

```
```powershell
git add README.md src/lib/tasks

git commit `
-m "feat: add task domain and SQLite repository" `
-m "Add task validation, overdue calculation, sorting, and database operations for creating,
editing, listing, and archiving tasks." `
-m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"

git push
```
```

That is now a meaningful commit containing both the backend task layer and the previously pending AI-review declaration.

Afterward, check:

```
```powershell
git status
git log -1 --format=full
```
```

The working tree should be clean, and the full commit output should show the `Assisted-by` line. Then we move directly to the Server Actions and create/list frontend.

User

Message 35

Ws lets continue

Assistant

Message 36

W. Now we build the first complete **vertical slice**: create a task through the frontend, save it in SQLite, and display it from the database.

We will use a Server Action rather than a separate API route. Next.js forms can call Server Actions directly, `revalidatePath("/")` refreshes the page after a mutation, and `connection()` ensures the SQLite-backed page renders at request time rather than being frozen during the production build.
([nextjs.org](https://nextjs.org/docs/app/guides/forms?utm_source=chatgpt.com))(https://nextjs.org/docs/app/guides/forms?utm_source=chatgpt.com))

1. Add action-state types

At the bottom of:

```
```text
src/lib/tasks/types.ts
```
```

add:

```
```typescript
export type TaskActionStatus =
 | "idle"
 | "success"
 | "error";

export interface TaskActionState {
 status: TaskActionStatus;
 message: string;
}
```
```

Your existing `src/lib/tasks/index.ts` already exports everything from `types.ts`, so no other export change is required.

2. Create the Server Action

Create:

```
```text
src/app/actions.ts
```
```

```
```typescript
"use server";
```

```
import { revalidatePath } from "next/cache";
```

```
import {
 addTask,
 isTaskStatus,
 TaskValidationError,
} from "@lib/tasks";
import type { TaskActionState } from "@lib/tasks/types";
```

```
export async function createTaskAction(
 _previousState: TaskActionState,
 formData: FormData,
): Promise<TaskActionState> {
 const status = String(formData.get("status")) ?? "";
```

```
 if (!isTaskStatus(status)) {
 return {
 status: "error",
 message: "The selected task status is invalid.",
 };
 }
}
```

```

try {
 addTask({
 title: String(formData.get("title") ?? ""),
 description: String(
 formData.get("description") ?? "",
),
 dueDate: String(formData.get("dueDate") ?? ""),
 topic: String(formData.get("topic") ?? ""),
 status,
 });

 revalidatePath("/");

 return {
 status: "success",
 message: "Task created successfully.",
 };
} catch (error) {
 if (error instanceof TaskValidationError) {
 return {
 status: "error",
 message: error.message,
 };
 }
}

console.error("Failed to create task:", error);

return {
 status: "error",
 message:
 "The task could not be created. Please try again.",
};
}
}
...

```

There is still no API route and no client-side database access. The Server Action receives the form submission and calls the repository-backed service.

---

### ## 3. Create the task form

Create:

```

```text
src/components/tasks/create-task-form.tsx
```

```tsx
"use client";

import { useActionState } from "react";

import { createTaskAction } from "@app/actions";
import { TASK_STATUSES } from "@lib/tasks/types";
import type { TaskActionState } from "@lib/tasks/types";

const initialState: TaskActionState = {
  status: "idle",
  message: "",
};

export function CreateTaskForm() {
  const [state, formAction, isPending] = useActionState(
    createTaskAction,
    initialState,
  );

  return (
    <form
      action={formAction}
      className="rounded-2xl border border-slate-200 bg-white p-6 shadow-sm"
    >
      <div className="mb-6">
        <p className="text-sm font-semibold uppercase tracking-wider text-blue-600">
          New task
        </p>

        <h2 className="mt-1 text-2xl font-bold text-slate-900">

```

Add something to your list

</h2>

<p className="mt-2 text-sm text-slate-600">

Enter the task information and choose its current status.

</p>

</div>

<div className="grid gap-5 md:grid-cols-2">

<div className="md:col-span-2">

<label

htmlFor="title"

className="mb-2 block text-sm font-semibold text-slate-700"

>

Title

</label>

<input

id="title"

name="title"

type="text"

required

maxLength={120}

placeholder="e.g. Complete SDP lab"

className="w-full rounded-xl border border-slate-300 px-4 py-3 text-slate-900

outline-none transition focus:border-blue-500 focus:ring-4 focus:ring-blue-100"

/>

</div>

<div className="md:col-span-2">

<label

htmlFor="description"

className="mb-2 block text-sm font-semibold text-slate-700"

>

Description

</label>

<textarea

id="description"

name="description"

rows={4}

maxLength={1000}

placeholder="Add any useful details about the task"

className="w-full resize-y rounded-xl border border-slate-300 px-4 py-3

text-slate-900 outline-none transition focus:border-blue-500 focus:ring-4

focus:ring-blue-100"

/>

</div>

<div>

<label

htmlFor="dueDate"

className="mb-2 block text-sm font-semibold text-slate-700"

>

Due date

</label>

<input

id="dueDate"

name="dueDate"

type="date"

required

className="w-full rounded-xl border border-slate-300 px-4 py-3 text-slate-900

outline-none transition focus:border-blue-500 focus:ring-4 focus:ring-blue-100"

/>

</div>

<div>

<label

htmlFor="topic"

className="mb-2 block text-sm font-semibold text-slate-700"

>

Topic

</label>

<input

id="topic"

name="topic"

type="text"

```

    required
    maxLength={80}
    placeholder="e.g. University"
    className="w-full rounded-xl border border-slate-300 px-4 py-3 text-slate-900
    outline-none transition focus:border-blue-500 focus:ring-4 focus:ring-blue-100"
  />
</div>

<div className="md:col-span-2">
  <label
    htmlFor="status"
    className="mb-2 block text-sm font-semibold text-slate-700"
  >
    Status
  </label>

  <select
    id="status"
    name="status"
    defaultValue="Todo"
    className="w-full rounded-xl border border-slate-300 bg-white px-4 py-3
    text-slate-900 outline-none transition focus:border-blue-500 focus:ring-4
    focus:ring-blue-100"
  >
    {TASK_STATUSES.map((status) => (
      <option key={status} value={status}>
        {status}
      </option>
    ))}
  </select>
</div>
</div>

{state.message && (
  <p
    role={state.status === "error" ? "alert" : "status"}
    className={`mt-5 rounded-xl px-4 py-3 text-sm font-medium ${
      state.status === "error"
        ? "bg-red-50 text-red-700"
        : "bg-emerald-50 text-emerald-700"
    }`}
  >
    {state.message}
  </p>
)}

<button
  type="submit"
  disabled={isPending}
  className="mt-6 w-full rounded-xl bg-blue-600 px-5 py-3 font-semibold text-white
  transition hover:bg-blue-700 disabled:cursor-not-allowed disabled:bg-blue-300"
>
  {isPending ? "Creating task..." : "Create task"}
</button>
</form>
);
}
` ``

```

React's `useState` supplies both the result returned by the action and a pending state for the submit button.
 ([react.dev](https://react.dev/reference/react/useState?utm_source=chatgpt.com))

4. Create the task list

Create:

```

` ``text
src/components/tasks/task-list.tsx
` ``

` ``tsx
import type { Task } from "@lib/tasks/types";

interface TaskListProps {
  tasks: Task[];
}

```

```

function formatDueDate(dueDate: string): string {
  const [year, month, day] = dueDate
    .split("-")
    .map(Number);

  return new Intl.DateTimeFormat("en-ZA", {
    dateStyle: "medium",
    timeZone: "UTC",
  }).format(
    new Date(Date.UTC(year, month - 1, day)),
  );
}

function statusClasses(status: Task["status"]): string {
  switch (status) {
    case "Todo":
      return "bg-slate-100 text-slate-700";

    case "In-Progress":
      return "bg-amber-100 text-amber-800";

    case "Complete":
      return "bg-emerald-100 text-emerald-800";
  }
}

export function TaskList({ tasks }: TaskListProps) {
  if (tasks.length === 0) {
    return (
      <div className="rounded-2xl border border-dashed border-slate-300 bg-white px-6 py-16 text-center">
        <div className="mx-auto flex h-14 w-14 items-center justify-center rounded-full bg-slate-100 text-2xl">
          ✓
        </div>

        <h2 className="mt-4 text-xl font-bold text-slate-900">
          No active tasks
        </h2>

        <p className="mt-2 text-sm text-slate-600">
          Create your first task using the form.
        </p>
      </div>
    );
  }

  return (
    <div className="space-y-4">
      {tasks.map((task) => (
        <article
          key={task.id}
          className={`rounded-2xl border bg-white p-5 shadow-sm ${
            task.isOverdue
              ? "border-red-300"
              : "border-slate-200"
          }`}
        >
          <div className="flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
            <div className="min-w-0">
              <div className="flex flex-wrap items-center gap-2">
                <span className="rounded-full bg-blue-50 px-3 py-1 text-xs font-semibold text-blue-700">
                  {task.topic}
                </span>

                <span
                  className={`rounded-full px-3 py-1 text-xs font-semibold ${statusClasses(
                    task.status,
                  )}`}
                >
                  {task.status}
                </span>

                {task.isOverdue && (
                  <span className="rounded-full bg-red-100 px-3 py-1 text-xs font-semibold text-red-700">
                    Overdue
                  </span>
                )}
            </div>
          </div>
        </article>
      )
    )}
    </div>
  );
}

```

```

</div>

<h3 className="mt-3 break-words text-xl font-bold text-slate-900">
  {task.title}
</h3>

{task.description ? (
  <p className="mt-2 whitespace-pre-wrap break-words text-sm leading-6
  text-slate-600">
    {task.description}
  </p>
) : (
  <p className="mt-2 text-sm italic text-slate-400">
    No description provided.
  </p>
)}
</div>

<div
  className={`shrink-0 rounded-xl px-4 py-3 text-sm ${
    task.isOverdue
      ? "bg-red-50 text-red-700"
      : "bg-slate-50 text-slate-600"
  }`}
>
  <p className="text-xs font-semibold uppercase tracking-wide">
    Due
  </p>

  <p className="mt-1 font-bold">
    {formatDueDate(task.dueDate)}
  </p>
</div>
</div>
</article>
  )}
</div>
);
}
...

```

Notice that `Overdue` is displayed as an additional visual badge but remains separate from the three status values, exactly as the brief requires. `filecite turn0file1`

5. Replace the homepage

Replace all of:

```

```text
src/app/page.tsx
```

```

with:

```

```tsx
import { connection } from "next/server";

import { CreateTaskForm } from "@components/tasks/create-task-form";
import { TaskList } from "@components/tasks/task-list";
import { getTasks } from "@lib/tasks";

export default async function Home() {
 await connection();

 const tasks = getTasks({
 archived: false,
 sortBy: "dueDate",
 });

 const completedTasks = tasks.filter(
 (task) => task.status === "Complete",
).length;

 const overdueTasks = tasks.filter(
 (task) => task.isOverdue,
).length;

 return (

```

```

<main className="min-h-screen bg-slate-100 px-4 py-10 sm:px-6 lg:px-8">
 <div className="mx-auto max-w-6xl">
 <header className="mb-8">
 <p className="text-sm font-bold uppercase tracking-[0.2em] text-blue-600">
 Local task manager
 </p>

 <h1 className="mt-2 text-4xl font-black tracking-tight text-slate-950 sm:text-5xl">
 My tasks
 </h1>

 <p className="mt-3 max-w-2xl text-base leading-7 text-slate-600">
 Create and organise tasks stored locally on this
 computer.
 </p>
 </header>

 <section className="mb-8 grid gap-4 sm:grid-cols-3">
 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Active tasks
 </p>

 <p className="mt-2 text-3xl font-black text-slate-900">
 {tasks.length}
 </p>
 </div>

 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Complete
 </p>

 <p className="mt-2 text-3xl font-black text-emerald-600">
 {completedTasks}
 </p>
 </div>

 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Overdue
 </p>

 <p className="mt-2 text-3xl font-black text-red-600">
 {overdueTasks}
 </p>
 </div>
 </section>

 <div className="grid gap-8 lg:grid-cols-[380px_1fr] lg:items-start">
 <aside className="lg:sticky lg:top-8">
 <CreateTaskForm />
 </aside>

 <section>
 <div className="mb-4 flex items-end justify-between gap-4">
 <div>
 <p className="text-sm font-semibold uppercase tracking-wider text-slate-500">
 Active list
 </p>

 <h2 className="mt-1 text-2xl font-bold text-slate-900">
 Tasks by due date
 </h2>
 </div>

 <TaskList tasks={tasks} />
 </div>
 </section>
 </div>
 </div>
</main>
);
}
...

```

---

### 6. Update the page metadata

Open:

```
```text
src/app/layout.tsx
```
```

Find the generated `metadata` and replace it with:

```
```typescript
export const metadata: Metadata = {
  title: "Local Task Manager",
  description:
    "A local-first task manager built with Next.js and SQLite.",
};
```
```

Leave the rest of `layout.tsx` unchanged.

---

### ## 7. Run and manually verify

Start the app:

```
```powershell
npm run dev
```
```

Create at least three temporary tasks:

1. A future `Todo` task.
2. A past `In-Progress` task—it should display **Overdue**.
3. A past `Complete` task—it should **not** display Overdue.

Then verify persistence:

```
```text
1. Reload the page.
2. Stop the server with Ctrl + C.
3. Start it again with npm run dev.
4. Confirm all three tasks remain.
```
```

The physical SQLite files should now exist:

```
```powershell
Get-ChildItem .\data
```
```

You will probably see:

```
```text
tasks.db
tasks.db-shm
tasks.db-wal
```
```

They should remain ignored by Git.

Finally run:

```
```powershell
npm run lint
npm run build
git status
```
```

The build output should now show `/` as a dynamic route rather than a static `/\_` route because `connection()` excludes the database-backed page from prerendering.  
([nextjs.org](https://nextjs.org/docs/app/api-reference/functions/connection?utm_source=chatgpt.com))([https://nextjs.org/docs/app/api-reference/functions/connection?utm\\_source=chatgpt.com](https://nextjs.org/docs/app/api-reference/functions/connection?utm_source=chatgpt.com)))

Do not commit yet—send me the result and a screenshot of the

User

Message 37

[Two screenshots uploaded]

PS C:\Code\SDP\coms-3011a-lab-1>

Get-ChildItem .\data

Directory: C:\Code\SDP\coms-3011a-lab-1\data

| Mode   | LastWriteTime    | Length | Name         |
|--------|------------------|--------|--------------|
| ----   | -----            | -----  | ----         |
| -a---- | 2026/08/01 19:39 | 4096   | tasks.db     |
| -a---- | 2026/08/01 19:48 | 32768  | tasks.db-shm |
| -a---- | 2026/08/01 19:46 | 119512 | tasks.db-wal |

```
PS C:\Code\SDP\coms-3011a-lab-1> npm run lint
>> npm run build
>> git status
```

```
> coms-3011a-lab-1@0.1.0 lint
> eslint
```

```
> coms-3011a-lab-1@0.1.0 build
> next build
```

▲ Next.js 16.2.12 (Turbopack)

Creating an optimized production build ...

- ✓ Compiled successfully in 3.0s
- ✓ Finished TypeScript in 3.1s
- ✓ Collecting page data using 5 workers in 869ms
- ✓ Generating static pages using 5 workers (4/4) in 1021ms
- ✓ Finalizing page optimization in 13ms

Route (app)

└─ f /  
└─ ○ /\_not-found

○ (Static) prerendered as static content  
f (Dynamic) server-rendered on demand

On branch main  
Your branch is up to date with 'origin/main'.

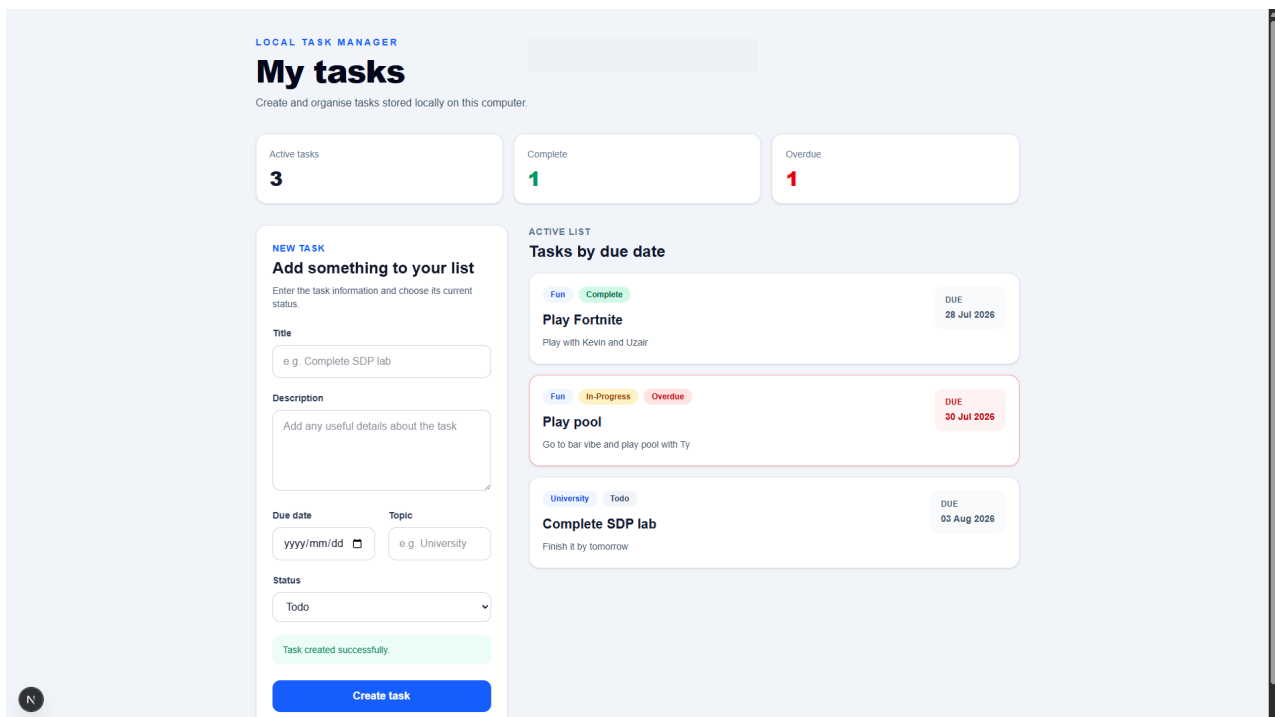
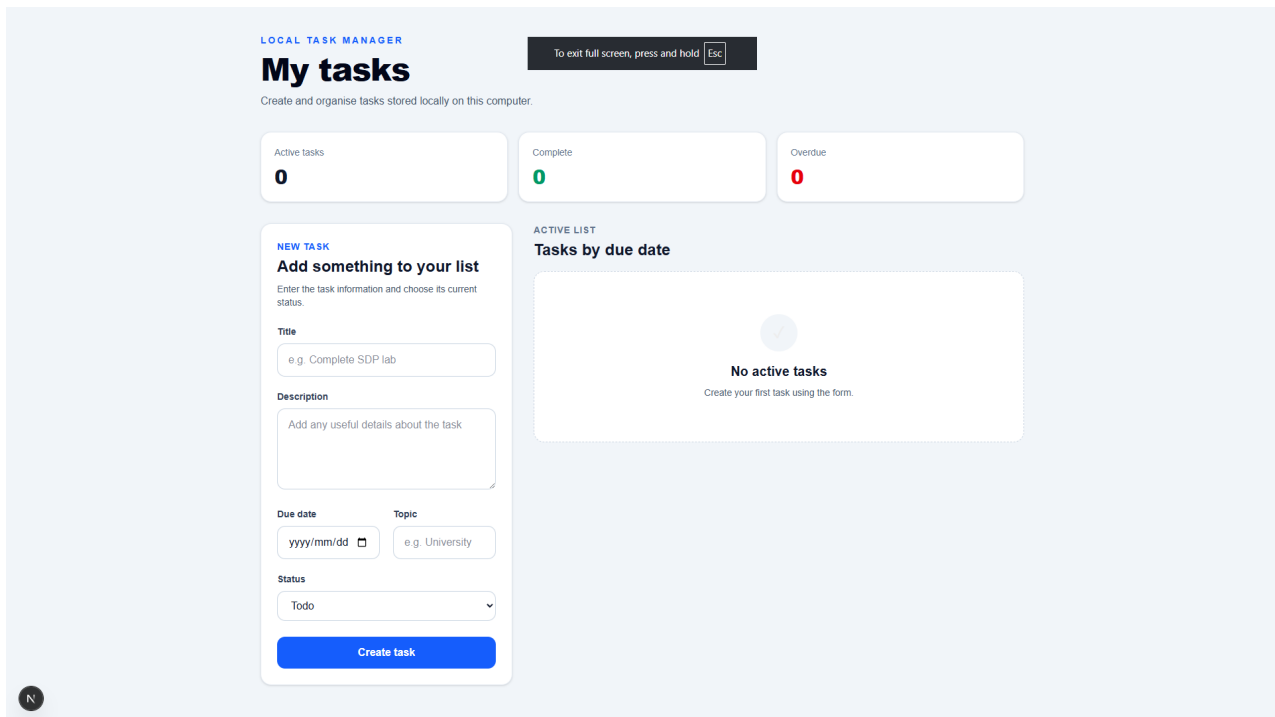
Changes not staged for commit:  
(use "git add <file>..." to update what will be committed)  
(use "git restore <file>..." to discard changes in working directory)

- modified: src/app/layout.tsx
- modified: src/app/page.tsx
- modified: src/lib/tasks/types.ts

Untracked files:  
(use "git add <file>..." to include in what will be committed)

- src/app/actions.ts
- src/components/

no changes added to commit (use "git add" and/or "git commit -a")



## Assistant

Message 38

W ☐ This slice is working properly.

From the screenshots and terminal output, we have verified:

- Tasks can be created with all four required fields.
- The tasks are stored in the real SQLite database.
- Data survives page reloads and server restarts.
- Tasks are currently ordered by due date.
- The past `In-Progress` task is marked **\*\*Overdue\*\***.
- The past `Complete` task is correctly **\*\*not\*\*** marked overdue.
- Overdue remains a visual indicator rather than a selectable status.
- The summary counters update correctly.
- `tasks.db`, `tasks.db-shm`, and `tasks.db-wal` were created and remain ignored.
- ESLint, TypeScript and the production build all pass.
- `/` is now dynamically rendered, which is correct for the SQLite-backed page.

That covers a meaningful portion of the functional walkthrough already. ☐filecite☐turn0file1☐

## Commit this vertical slice

```
```powershell
git add src/app src/components src/lib/tasks/types.ts

git commit `
-m "feat: add task creation and active task list" `
-m "Connect the task form to SQLite using a Server Action and display persisted tasks with
status, due date, and overdue indicators." `
-m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"

git push
```
```

Then confirm:

```
```powershell
git status
git log -1 --format=full
```
```

This should be commit 5 overall. The next slice will complete the main application:

- Sort selector for topic, status and due date.
- Edit task form.
- Archive action.
- Active and archived views.
- Archived tasks remain visible but cannot be edited.
- Preserve sorting through URL query parameters.

One small observation: the UI is already looking clean and readable, so we do not need to waste time redesigning it. We should now focus directly on the remaining rubric functionality.

**User**

Message 39

ok Ws is there anything left for this session or must we still continue dont code just tell me what we have to do left

**Assistant**

Message 40

We should still continue this session. The create/list flow works, but the core frontend/backend is not finished yet.

What remains for today:

- **\*\*Edit tasks\*\*** and persist the changes after reload.
- **\*\*Archive tasks\*\*** without deleting them.
- Add an **\*\*archived-task view\*\*** so archived items remain accessible.
- Add sorting by:
  - Topic
  - Status
  - Due date
- Make sure archived tasks cannot accidentally be edited or archived again.
- Manually verify all of those behaviours.
- Commit that as one final coherent feature slice.

That would complete nearly all seven functional walkthrough requirements except the formal clean-clone check and automated tests. `filecite` `turn0file1`

Then the final session can focus on:

- At least three deterministic tests using a throwaway database.
- One test covering archive or overdue behaviour.
- Final README sections: Third-Party Code, Database Design and Running It.
- AI transcripts and declarations.
- Full clean-clone walkthrough.
- Any final polish or bug fixes.

We probably do **\*\*not\*\*** need separate API routes at all because Server Actions already connect the frontend to the backend cleanly. So today's remaining feature slice should finish the functional application.

**User**

Message 41

ok lets continue

**Assistant**

Message 42

W. This is the **final functional slice** for session 2: editing, archiving, archived-task viewing and all three sorting options. We'll keep Server Actions rather than introducing API routes; Next.js supports forms calling Server Actions directly, and `revalidatePath` refreshes the affected page after mutations. In Next.js 16, page `searchParams` are asynchronous, so the homepage below awaits them.  
([nextjs.org](https://nextjs.org/docs/app/guides/migrating-to-cache-components?utm_source=chatgpt.com))([https://nextjs.org/docs/app/guides/migrating-to-cache-components?utm\\_source=chatgpt.com](https://nextjs.org/docs/app/guides/migrating-to-cache-components?utm_source=chatgpt.com)))

### ## 1. Export the not-found error

At the bottom of:

```
```text
src/lib/tasks/index.ts
```
```

add:

```
```typescript
export { TaskNotFoundError } from "../repository";
```
```

The end of the file should now look like:

```
```typescript
export * from "../types";

export {
  isTaskOverdue,
  isTaskSortOption,
  isTaskStatus,
  isValidDateString,
  TaskValidationError,
} from "../validation";

export { TaskNotFoundError } from "../repository";
```
```

---

### ## 2. Replace the Server Actions

Replace all of:

```
```text
src/app/actions.ts
```
```

with:

```
```typescript
"use server";

import { revalidatePath } from "next/cache";

import {
  addTask,
  archiveTask as archiveTaskInDatabase,
  editTask,
  isTaskStatus,
  TaskNotFoundError,
  TaskValidationError,
} from "@lib/tasks";

import type {
  TaskActionState,
  TaskInput,
} from "@lib/tasks/types";

function readTaskId(formData: FormData): number | null {
  const id = Number(formData.get("taskId"));

  if (!Number.isInteger(id) || id <= 0) {
    return null;
  }

  return id;
}

function readTaskInput(
```

```

    formData: FormData,
  ): TaskInput | null {
    const status = String(formData.get("status") ?? "");

    if (!isTaskStatus(status)) {
      return null;
    }

    return {
      title: String(formData.get("title") ?? ""),
      description: String(
        formData.get("description") ?? "",
      ),
      dueDate: String(formData.get("dueDate") ?? ""),
      topic: String(formData.get("topic") ?? ""),
      status,
    };
  }

function handleTaskError(
  error: unknown,
  fallbackMessage: string,
  consoleMessage: string,
): TaskActionState {
  if (
    error instanceof TaskValidationError ||
    error instanceof TaskNotFoundError
  ) {
    return {
      status: "error",
      message: error.message,
    };
  }

  console.error(consoleMessage, error);

  return {
    status: "error",
    message: fallbackMessage,
  };
}

export async function createTaskAction(
  _previousState: TaskActionState,
  formData: FormData,
): Promise<TaskActionState> {
  const input = readTaskInput(formData);

  if (!input) {
    return {
      status: "error",
      message: "The selected task status is invalid.",
    };
  }

  try {
    addTask(input);

    revalidatePath("/");

    return {
      status: "success",
      message: "Task created successfully.",
    };
  } catch (error) {
    return handleTaskError(
      error,
      "The task could not be created. Please try again.",
      "Failed to create task:",
    );
  }
}

export async function editTaskAction(
  _previousState: TaskActionState,
  formData: FormData,
): Promise<TaskActionState> {
  const taskId = readTaskId(formData);
  const input = readTaskInput(formData);

```

```

if (!taskId) {
  return {
    status: "error",
    message: "The task identifier is invalid.",
  };
}

if (!input) {
  return {
    status: "error",
    message: "The selected task status is invalid.",
  };
}

try {
  editTask(taskId, input);

  revalidatePath("/");

  return {
    status: "success",
    message: "Task updated successfully.",
  };
} catch (error) {
  return handleTaskError(
    error,
    "The task could not be updated. Please try again.",
    "Failed to update task:",
  );
}
}

export async function archiveTaskAction(
  _previousState: TaskActionState,
  formData: FormData,
): Promise<TaskActionState> {
  const taskId = readTaskId(formData);

  if (!taskId) {
    return {
      status: "error",
      message: "The task identifier is invalid.",
    };
  }

  try {
    archiveTaskInDatabase(taskId);

    revalidatePath("/");

    return {
      status: "success",
      message: "Task archived successfully.",
    };
  } catch (error) {
    return handleTaskError(
      error,
      "The task could not be archived. Please try again.",
      "Failed to archive task:",
    );
  }
}

```

There is still deliberately no delete action.

3. Create the task card

Create:

```

```text
src/components/tasks/task-card.tsx
```

```

Add:

```

```tsx
"use client";

```

```

import { useActionState, useState } from "react";

import {
 archiveTaskAction,
 editTaskAction,
} from "@app/actions";

import {
 TASK_STATUSES,
 type Task,
 type TaskActionState,
} from "@lib/tasks/types";

interface TaskCardProps {
 task: Task;
}

const initialState: TaskActionState = {
 status: "idle",
 message: "",
};

function formatDueDate(dueDate: string): string {
 const [year, month, day] = dueDate
 .split("-")
 .map(Number);

 return new Intl.DateTimeFormat("en-ZA", {
 dateStyle: "medium",
 timeZone: "UTC",
 }).format(
 new Date(Date.UTC(year, month - 1, day)),
);
}

function formatArchivedDate(timestamp: string): string {
 const normalisedTimestamp = timestamp.includes("T")
 ? timestamp
 : `${timestamp.replace(" ", "T")}Z`;

 const date = new Date(normalisedTimestamp);

 if (Number.isNaN(date.getTime())) {
 return timestamp;
 }

 return new Intl.DateTimeFormat("en-ZA", {
 dateStyle: "medium",
 timeStyle: "short",
 }).format(date);
}

function statusClasses(status: Task["status"]): string {
 switch (status) {
 case "Todo":
 return "bg-slate-100 text-slate-700";

 case "In-Progress":
 return "bg-amber-100 text-amber-800";

 case "Complete":
 return "bg-emerald-100 text-emerald-800";
 }
}

export function TaskCard({ task }: TaskCardProps) {
 const [isEditing, setIsEditing] = useState(false);

 const [
 editState,
 editFormAction,
 isEditPending,
] = useActionState(editTaskAction, initialState);

 const [
 archiveState,
 archiveFormAction,
 isArchivePending,
] = useActionState(archiveTaskAction, initialState);

```

```

const isArchived = task.archivedAt !== null;

if (isEditing && !isArchived) {
 return (
 <article className="rounded-2xl border border-blue-300 bg-white p-5 shadow-sm">
 <div className="mb-5">
 <p className="text-sm font-semibold uppercase tracking-wider text-blue-600">
 Editing task
 </p>

 <h3 className="mt-1 text-xl font-bold text-slate-900">
 Update task #{task.id}
 </h3>
 </div>

 <form action={editFormAction}>
 <input
 type="hidden"
 name="taskId"
 value={task.id}
 />

 <div className="grid gap-4 sm:grid-cols-2">
 <div className="sm:col-span-2">
 <label
 htmlFor={`edit-title-${task.id}`}
 className="mb-2 block text-sm font-semibold text-slate-700"
 >
 Title
 </label>

 <input
 id={`edit-title-${task.id}`}
 name="title"
 type="text"
 required
 maxLength={120}
 defaultValue={task.title}
 className="w-full rounded-xl border border-slate-300 px-4 py-3 text-slate-900
 outline-none transition focus:border-blue-500 focus:ring-4
 focus:ring-blue-100"
 />
 </div>

 <div className="sm:col-span-2">
 <label
 htmlFor={`edit-description-${task.id}`}
 className="mb-2 block text-sm font-semibold text-slate-700"
 >
 Description
 </label>

 <textarea
 id={`edit-description-${task.id}`}
 name="description"
 rows={3}
 maxLength={1000}
 defaultValue={task.description}
 className="w-full resize-y rounded-xl border border-slate-300 px-4 py-3
 text-slate-900 outline-none transition focus:border-blue-500 focus:ring-4
 focus:ring-blue-100"
 />
 </div>

 <div>
 <label
 htmlFor={`edit-due-date-${task.id}`}
 className="mb-2 block text-sm font-semibold text-slate-700"
 >
 Due date
 </label>

 <input
 id={`edit-due-date-${task.id}`}
 name="dueDate"
 type="date"
 required
 defaultValue={task.dueDate}
 className="w-full rounded-xl border border-slate-300 px-4 py-3 text-slate-900

```

```

 outline-none transition focus:border-blue-500 focus:ring-4
 focus:ring-blue-100"
 />
 </div>

 <div>
 <label
 htmlFor={`edit-topic-${task.id}`}
 className="mb-2 block text-sm font-semibold text-slate-700"
 >
 Topic
 </label>

 <input
 id={`edit-topic-${task.id}`}
 name="topic"
 type="text"
 required
 maxLength={80}
 defaultValue={task.topic}
 className="w-full rounded-xl border border-slate-300 px-4 py-3 text-slate-900
 outline-none transition focus:border-blue-500 focus:ring-4
 focus:ring-blue-100"
 />
 </div>

 <div className="sm:col-span-2">
 <label
 htmlFor={`edit-status-${task.id}`}
 className="mb-2 block text-sm font-semibold text-slate-700"
 >
 Status
 </label>

 <select
 id={`edit-status-${task.id}`}
 name="status"
 defaultValue={task.status}
 className="w-full rounded-xl border border-slate-300 bg-white px-4 py-3
 text-slate-900 outline-none transition focus:border-blue-500 focus:ring-4
 focus:ring-blue-100"
 >
 {TASK_STATUSES.map((status) => (
 <option key={status} value={status}>
 {status}
 </option>
))}
 </select>
 </div>
 </div>

 {editState.message && (
 <p
 role={
 editState.status === "error"
 ? "alert"
 : "status"
 }
 className={`mt-4 rounded-xl px-4 py-3 text-sm font-medium ${
 editState.status === "error"
 ? "bg-red-50 text-red-700"
 : "bg-emerald-50 text-emerald-700"
 }`}
 >
 {editState.message}
 </p>
)}

 <div className="mt-5 flex flex-col gap-3 sm:flex-row sm:justify-end">
 <button
 type="button"
 disabled={isEditPending}
 onClick={() => setIsEditing(false)}
 className="rounded-xl border border-slate-300 px-5 py-2.5 font-semibold
 text-slate-700 transition hover:bg-slate-50 disabled:cursor-not-allowed
 disabled:opacity-50"
 >
 Cancel
 </button>
 </div>

```

```

 <button
 type="submit"
 disabled={isEditPending}
 className="rounded-xl bg-blue-600 px-5 py-2.5 font-semibold text-white
 transition hover:bg-blue-700 disabled:cursor-not-allowed disabled:bg-blue-300"
 >
 {isEditPending
 ? "Saving changes..."
 : "Save changes"}
 </button>
 </div>
</form>
</article>
);
}

return (
 <article
 className={`rounded-2xl border bg-white p-5 shadow-sm ${
 task.isOverdue
 ? "border-red-300"
 : "border-slate-200"
 }`}
 >
 <div className="flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
 <div className="min-w-0">
 <div className="flex flex-wrap items-center gap-2">
 <span className="rounded-full bg-blue-50 px-3 py-1 text-xs font-semibold
 text-blue-700">
 {task.topic}

 <span
 className={`rounded-full px-3 py-1 text-xs font-semibold ${statusClasses(
 task.status,
)}`}
 >
 {task.status}

 {task.isOverdue && (
 <span className="rounded-full bg-red-100 px-3 py-1 text-xs font-semibold
 text-red-700">
 Overdue

)}

 {isArchived && (
 <span className="rounded-full bg-purple-100 px-3 py-1 text-xs font-semibold
 text-purple-700">
 Archived

)}
 </div>

 <h3 className="mt-3 break-words text-xl font-bold text-slate-900">
 {task.title}
 </h3>

 {task.description ? (
 <p className="mt-2 whitespace-pre-wrap break-words text-sm leading-6
 text-slate-600">
 {task.description}
 </p>
) : (
 <p className="mt-2 text-sm italic text-slate-400">
 No description provided.
 </p>
)}

 {task.archivedAt && (
 <p className="mt-3 text-xs font-medium text-purple-700">
 Archived{" "}
 {formatArchivedDate(task.archivedAt)}
 </p>
)}
 </div>

 <div
 className={`shrink-0 rounded-xl px-4 py-3 text-sm ${

```

```

 task.isOverdue
 ? "bg-red-50 text-red-700"
 : "bg-slate-50 text-slate-600"
 }` }
 >
 <p className="text-xs font-semibold uppercase tracking-wide">
 Due
 </p>

 <p className="mt-1 font-bold">
 {formatDueDate(task.dueDate)}
 </p>
</div>
</div>

{!isArchived && (
 <div className="mt-5 border-t border-slate-100 pt-4">
 <div className="flex flex-col gap-3 sm:flex-row sm:justify-end">
 <button
 type="button"
 onClick={() => setIsEditing(true)}
 className="rounded-xl border border-blue-200 px-4 py-2 text-sm font-semibold
 text-blue-700 transition hover:bg-blue-50"
 >
 Edit
 </button>

 <form action={archiveFormAction}>
 <input
 type="hidden"
 name="taskId"
 value={task.id}
 />

 <button
 type="submit"
 disabled={isArchivePending}
 className="w-full rounded-xl border border-purple-200 px-4 py-2 text-sm
 font-semibold text-purple-700 transition hover:bg-purple-50
 disabled:cursor-not-allowed disabled:opacity-50"
 >
 {isArchivePending
 ? "Archiving..."
 : "Archive"}
 </button>
 </form>
 </div>

 {archiveState.message && (
 <p
 role={
 archiveState.status === "error"
 ? "alert"
 : "status"
 }
 className={`mt-3 rounded-xl px-4 py-3 text-sm font-medium ${
 archiveState.status === "error"
 ? "bg-red-50 text-red-700"
 : "bg-emerald-50 text-emerald-700"
 }`}
 >
 {archiveState.message}
 </p>
)}
 </div>
)}
</article>
);
}
...

```

Archived tasks have no edit or archive controls. There is still no deletion functionality.

---

#### ## 4. Replace the task list

Replace all of:

```
``text
```

```

src/components/tasks/task-list.tsx
...

with:

```tsx
import { TaskCard } from "../task-card";

import type { Task } from "@lib/tasks/types";

interface TaskListProps {
  tasks: Task[];
  archived: boolean;
}

export function TaskList({
  tasks,
  archived,
}: TaskListProps) {
  if (tasks.length === 0) {
    return (
      <div className="rounded-2xl border border-dashed border-slate-300 bg-white px-6 py-16 text-center">
        <div className="mx-auto flex h-14 w-14 items-center justify-center rounded-full bg-slate-100 text-2xl">
          {archived ? "□" : "✓"}
        </div>

        <h2 className="mt-4 text-xl font-bold text-slate-900">
          {archived
            ? "No archived tasks"
            : "No active tasks"}
        </h2>

        <p className="mt-2 text-sm text-slate-600">
          {archived
            ? "Tasks you archive will remain viewable here."
            : "Create your first task using the form."}
        </p>
      </div>
    );
  }

  return (
    <div className="space-y-4">
      {tasks.map((task) => (
        <TaskCard
          key={[
            task.id,
            task.title,
            task.description,
            task.dueDate,
            task.topic,
            task.status,
            task.archivedAt,
          ].join("-")}
          task={task}
        </>
      ))}
    </div>
  );
}
...

```

The detailed key causes the task card to remount after a successful edit, closing the edit form and showing the refreshed task values.

5. Replace the homepage

Replace all of:

```

```text
src/app/page.tsx
```

```

with:

```

```tsx

```

```

import Link from "next/link";
import { connection } from "next/server";

import { CreateTaskForm } from "@components/tasks/create-task-form";
import { TaskList } from "@components/tasks/task-list";

import {
 getTasks,
 isTaskSortOption,
 type TaskSortOption,
} from "@lib/tasks";

interface HomeProps {
 searchParams: Promise<{
 view?: string | string[];
 sort?: string | string[];
 }>;
}

type TaskView = "active" | "archived";

const SORT_LABELS: Record<TaskSortOption, string> = {
 topic: "topic",
 status: "status",
 dueDate: "due date",
};

function firstQueryValue(
 value: string | string[] | undefined,
): string | undefined {
 return Array.isArray(value) ? value[0] : value;
}

export default async function Home({
 searchParams,
}: HomeProps) {
 await connection();

 const parameters = await searchParams;

 const requestedView = firstQueryValue(parameters.view);
 const requestedSort = firstQueryValue(parameters.sort);

 const view: TaskView =
 requestedView === "archived"
 ? "archived"
 : "active";

 const sortBy: TaskSortOption =
 requestedSort && isTaskSortOption(requestedSort)
 ? requestedSort
 : "dueDate";

 const activeTasks = getTasks({
 archived: false,
 sortBy,
 });

 const archivedTasks = getTasks({
 archived: true,
 sortBy,
 });

 const visibleTasks =
 view === "archived"
 ? archivedTasks
 : activeTasks;

 const completedTasks = activeTasks.filter(
 (task) => task.status === "Complete",
).length;

 const overdueTasks = activeTasks.filter(
 (task) => task.isOverdue,
).length;

 const activeHref =
 `/?view=active&sort=${sortBy}`;

 const archivedHref =

```

```
`/?view=archived&sort=${sortBy}`;

return (
 <main className="min-h-screen bg-slate-100 px-4 py-10 sm:px-6 lg:px-8">
 <div className="mx-auto max-w-6xl">
 <header className="mb-8">
 <p className="text-sm font-bold uppercase tracking-[0.2em] text-blue-600">
 Local task manager
 </p>

 <h1 className="mt-2 text-4xl font-black tracking-tight text-slate-950 sm:text-5xl">
 My tasks
 </h1>

 <p className="mt-3 max-w-2xl text-base leading-7 text-slate-600">
 Create and organise tasks stored locally on this
 computer.
 </p>
 </header>

 <section className="mb-8 grid gap-4 sm:grid-cols-2 lg:grid-cols-4">
 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Active tasks
 </p>

 <p className="mt-2 text-3xl font-black text-slate-900">
 {activeTasks.length}
 </p>
 </div>

 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Complete
 </p>

 <p className="mt-2 text-3xl font-black text-emerald-600">
 {completedTasks}
 </p>
 </div>

 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Overdue
 </p>

 <p className="mt-2 text-3xl font-black text-red-600">
 {overdueTasks}
 </p>
 </div>

 <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
 <p className="text-sm font-medium text-slate-500">
 Archived
 </p>

 <p className="mt-2 text-3xl font-black text-purple-600">
 {archivedTasks.length}
 </p>
 </div>
 </section>

 <div className="grid gap-8 lg:grid-cols-[380px_1fr] lg:items-start">
 <aside className="lg:sticky lg:top-8">
 <CreateTaskForm />
 </aside>

 <section>
 <div className="mb-5 rounded-2xl border border-slate-200 bg-white p-4 shadow-sm">
 <div className="flex flex-col gap-4 xl:flex-row xl:items-end
 xl:justify-between">
 <nav
 aria-label="Task views"
 className="inline-flex w-fit rounded-xl bg-slate-100 p-1"
 >
 <Link
 href={activeHref}
 className={`rounded-lg px-4 py-2 text-sm font-semibold transition ${
 view === "active"
 ? "bg-white text-blue-700 shadow-sm"

```

```

 : "text-slate-600 hover:text-slate-900"
 }` }
 >
 Active ({activeTasks.length})
</Link>

<Link
 href={archivedHref}
 className={`rounded-lg px-4 py-2 text-sm font-semibold transition ${
 view === "archived"
 ? "bg-white text-purple-700 shadow-sm"
 : "text-slate-600 hover:text-slate-900"
 }` }
 >
 Archived ({archivedTasks.length})
</Link>
</nav>

<form
 method="get"
 className="flex flex-col gap-2 sm:flex-row sm:items-end"
 >
 <input
 type="hidden"
 name="view"
 value={view}
 />

 <div>
 <label
 htmlFor="sort"
 className="mb-1 block text-xs font-semibold uppercase tracking-wide text-slate-500"
 >
 Sort by
 </label>

 <select
 id="sort"
 name="sort"
 defaultValue={sortBy}
 className="w-full rounded-xl border border-slate-300 bg-white px-4 py-2.5 text-sm font-semibold text-slate-700 outline-none transition focus:border-blue-500 focus:ring-4 focus:ring-blue-100 sm:w-44"
 >
 <option value="dueDate">
 Due date
 </option>

 <option value="topic">
 Topic
 </option>

 <option value="status">
 Status
 </option>
 </select>
 </div>

 <button
 type="submit"
 className="rounded-xl bg-slate-900 px-4 py-2.5 text-sm font-semibold text-white transition hover:bg-slate-700"
 >
 Apply
 </button>
</form>
</div>

<div className="mb-4">
 <p className="text-sm font-semibold uppercase tracking-wider text-slate-500">
 {view === "archived"
 ? "Archive"
 : "Active list"}
 </p>

 <h2 className="mt-1 text-2xl font-bold text-slate-900">
 {view === "archived"
 ? "Archived tasks"

```

```

 : `Tasks by ${SORT_LABELS[sortBy]}` }
 </h2>
 </div>

 <TaskList
 tasks={visibleTasks}
 archived={view === "archived"}
 />
</section>
</div>
</div>
</main>
);
}
...

```

This completes the functional requirements for edit, archive, archived visibility and sorting by topic, status and due date. `filecite` `turn0file1`

## ## 6. Manually verify everything

Run:

```

```powershell
npm run dev
```

```

Test these in order:

1. **Due-date sorting:** your current tasks should appear earliest to latest.
2. **Topic sorting:** choose Topic and click **Apply**. Topics should sort alphabetically.
3. **Status sorting:** choose Status and click **Apply**. The order should be ``Todo``, then ``In-Progress``, then ``Complete``.
4. Click **Edit** on a task.
5. Change its title, topic, date and status, then save.
6. Reload the browser and confirm the edit remains.
7. Archive that task.
8. Confirm it immediately leaves the active list.
9. Open the **Archived** tab.
10. Confirm the task remains viewable and has no Edit or Archive buttons.
11. Stop and restart the development server and confirm all data remains.

Then run:

```

```powershell
npm run lint
npm run build
git status
```

```

Do not commit this slice until those behaviours pass. Send the terminal output and screenshots of the active and archived views.

## User

Message 43

[Two screenshots uploaded]

i tested everything it all works fine heres the terminal : PS C:\Code\SDP\coms-3011a-lab-1>

```

npm run lint
>> npm run build
>> git status

```

```

> coms-3011a-lab-1@0.1.0 lint
> eslint

```

```

> coms-3011a-lab-1@0.1.0 build
> next build

```

▲ Next.js 16.2.12 (Turbopack)

```

 Creating an optimized production build ...
 ✓ Compiled successfully in 2.9s
 ✓ Finished TypeScript in 3.2s
 ✓ Collecting page data using 5 workers in 899ms
 ✓ Generating static pages using 5 workers (4/4) in 1010ms
 ✓ Finalizing page optimization in 16ms

```

Route (app)

[ f /  
 ○ /\_not-found

- (Static) prerendered as static content
- f (Dynamic) server-rendered on demand

On branch main  
Your branch is up to date with 'origin/main'.

Changes not staged for commit:  
(use "git add <file>..." to update what will be committed)  
(use "git restore <file>..." to discard changes in working directory)  
modified: src/app/actions.ts  
modified: src/app/page.tsx  
modified: src/components/tasks/task-list.tsx  
modified: src/lib/tasks/index.ts

Untracked files:  
(use "git add <file>..." to include in what will be committed)  
src/components/tasks/task-card.tsx

no changes added to commit (use "git add" and/or "git commit -a")

LOCAL TASK MANAGER

# My tasks

Create and organise tasks stored locally on this computer.

Active tasks

2

Complete

0

Overdue

1

Archived

1

NEW TASK

**Add something to your list**  
Enter the task information and choose its current status.

Title

e.g. Complete SDP lab

Description

Add any useful details about the task

Due date

yyyy/mm/dd

Topic

e.g. University

Status

Todo

Create task

Active (2)

Archived (1)

SORT BY

Due date

Apply

ACTIVE LIST

**Tasks by due date**

Chore

In-Progress

Overdue

**Play pool**

Go to bar vibe and play pool with Ty

Edit

Archive

DUE

30 Jul 2026

University

Todo

**Complete SDP lab**

Finish it by tomorrow

Edit

Archive

DUE

03 Aug 2026

COMS3011A Lab 1 - Chat Transcript | Page 51

LOCAL TASK MANAGER

# My tasks

Create and organise tasks stored locally on this computer.

Active tasks  
**2**

Complete  
**0**

Overdue  
**1**

Archived  
**1**

NEW TASK

Add something to your list

Enter the task information and choose its current status.

Title

e.g. Complete SDP lab

Description

Add any useful details about the task

Due date

yyyy/mm/dd

Topic

e.g. University

Status

Todo

Create task

Active (2) Archived (1)

SORT BY

Due date

Apply

ARCHIVE

Archived tasks

Fun Complete Archived

Play Fortnite

Play with Kevin and Uzair

Archived 01 Aug 2026, 20:09

DUE

28 Jul 2026

## Assistant

Message 44

W ☐ Everything shown is correct.

The screenshots and terminal output confirm:

- Editing works and persists.
- Archiving removes the task from the active list.
- Archived tasks remain viewable.
- Archived tasks have no edit/archive controls.
- Sorting controls are present for due date, topic and status.
- Overdue remains a visual badge rather than a status.
- Active, completed, overdue and archived counts update correctly.
- Lint, TypeScript and the production build all pass.
- The page remains dynamically rendered for SQLite data.

This completes the functional application requirements from the brief. ☐filecite☐turn0file1☐

## Commit the final feature slice

```

`powershell
git add src/app/actions.ts `
 src/app/page.tsx `
 src/components/tasks/task-list.tsx `
 src/components/tasks/task-card.tsx `
 src/lib/tasks/index.ts

git commit `
-m "feat: add task editing archiving and sorting" `
-m "Allow active tasks to be edited and archived, provide an archived-task view, and support
 sorting by topic, status, and due date." `
-m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"

git push
`

```

Then verify:

```

`powershell
git status
git log --oneline --decorate -8
`

```

This should bring the repository to at least **\*\*six commits\*\***, satisfying the rubric's minimum commit count, with the work visibly spread across multiple sessions.

Session 2 is complete after that commit. The final session only needs:

- Automated tests against a throwaway database.

- Final README documentation.
- AI transcript preparation.
- Clean-clone installation and seven-step walkthrough.
- Any bugs discovered during final verification.

Separate API routes are not necessary unless testing reveals a genuine reason to add them.

**User**

Message 45

ok Ws session 2 is done ill see u tomorrow for the final session

**Assistant**

Message 46

W ☐ Session 2 is officially done.

You now have the full functional app working: create, edit, archive, archived view, sorting, overdue handling, persistence, and a clean passing build.

Tomorrow we finish tests, documentation, AI transcripts, and the clean-clone walkthrough.

**User**

Message 47

ok its the final session lets get started

**Assistant**

Message 48

W, final session ☐ We'll finish in this order:

1. Automated tests against a disposable SQLite database.
2. Final README documentation.
3. AI transcript/declaration check.
4. Clean-clone walkthrough and final submission audit.

The biggest remaining risk is testing, so we start there. The rubric requires at least three meaningful tests, including archiving or overdue behaviour, all running from one documented command against a throwaway database. ☐filecite☐turn0file1☐

## 1. Confirm the repository is clean

```
```powershell
git status
git log --oneline --decorate -10
```
```

Assuming the working tree is clean, continue.

## 2. Install Vitest

```
```powershell
npm install --save-dev vitest
```
```

Vitest supports Node 20 or newer, can test backend code, and uses a Node environment by default, so it suits our repository and SQLite layer without browser dependencies. ([vitest.dev](https://vitest.dev/guide/?utm\_source=chatgpt.com))

In `package.json`, add this inside `"scripts"`:

```
```json
"test": "vitest run"
```
```

Your scripts will resemble:

```
```json
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint",
  "test": "vitest run"
}
```
```

The single marking command will therefore be:

```
```powershell
npm test
```
```

```

3. Add the repository tests

Create:

```
```text
src/lib/tasks/repository.test.ts
```
```

Add:

```
```typescript
import type Database from "better-sqlite3";
import {
 afterEach,
 beforeEach,
 describe,
 expect,
 it,
} from "vitest";

import { openDatabase } from "../db/database";

import {
 archiveTask,
 createTask,
 listTasks,
 updateTask,
} from "../repository";

import type { TaskInput } from "../types";

let database: Database.Database;

function makeTask(
 overrides: Partial<TaskInput> = {},
): TaskInput {
 return {
 title: "Complete the lab",
 description: "Finish the remaining SDP work.",
 dueDate: "2026-08-10",
 topic: "University",
 status: "Todo",
 ...overrides,
 };
}

beforeEach(() => {
 database = openDatabase(":memory:");
});

afterEach(() => {
 database.close();
});

describe("task repository", () => {
 it("creates and retrieves a task with normalised input", () => {
 const createdTask = createTask(
 database,
 makeTask({
 title: " Complete the lab ",
 description: " Finish the remaining work. ",
 topic: " University ",
 }),
);

 expect(createdTask.id).toBeGreaterThan(0);
 expect(createdTask).toMatchObject({
 title: "Complete the lab",
 description: "Finish the remaining work.",
 dueDate: "2026-08-10",
 topic: "University",
 status: "Todo",
 archivedAt: null,
 });

 const activeTasks = listTasks(database);

 expect(activeTasks).toHaveLength(1);
 });
});
```
```

```

    expect(activeTasks[0].id).toBe(createdTask.id);
  });

it("updates an existing active task", () => {
  const createdTask = createTask(
    database,
    makeTask(),
  );

  const updatedTask = updateTask(
    database,
    createdTask.id,
    makeTask({
      title: "Submit the lab",
      description: "Upload the final repository.",
      dueDate: "2026-08-04",
      topic: "SDP",
      status: "In-Progress",
    }),
  );

  expect(updatedTask).toMatchObject({
    id: createdTask.id,
    title: "Submit the lab",
    description: "Upload the final repository.",
    dueDate: "2026-08-04",
    topic: "SDP",
    status: "In-Progress",
    archivedAt: null,
  });

  const storedTask = listTasks(database)[0];

  expect(storedTask).toMatchObject({
    title: "Submit the lab",
    topic: "SDP",
    status: "In-Progress",
  });
});

it("archives a task without deleting it", () => {
  const archivedCandidate = createTask(
    database,
    makeTask({
      title: "Archive this task",
    }),
  );

  const remainingTask = createTask(
    database,
    makeTask({
      title: "Keep this task active",
    }),
  );

  const archivedTask = archiveTask(
    database,
    archivedCandidate.id,
  );

  expect(archivedTask.archivedAt).not.toBeNull();

  const activeTasks = listTasks(database, {
    archived: false,
  });

  const archivedTasks = listTasks(database, {
    archived: true,
  });

  expect(activeTasks.map((task) => task.id)).toEqual([
    remainingTask.id,
  ]);

  expect(archivedTasks).toHaveLength(1);
  expect(archivedTasks[0].id).toBe(
    archivedCandidate.id,
  );
  expect(archivedTasks[0].title).toBe(
    "Archive this task",
  );

```

```

    );
  });

it("derives overdue state from the due date and status", () => {
  const currentDate = new Date(
    2026,
    7,
    2,
    12,
    0,
    0,
  );

  const overdueTask = createTask(
    database,
    makeTask({
      title: "Past todo task",
      dueDate: "2026-08-01",
      status: "Todo",
    }),
  );

  const completedTask = createTask(
    database,
    makeTask({
      title: "Past completed task",
      dueDate: "2026-08-01",
      status: "Complete",
    }),
  );

  const todayTask = createTask(
    database,
    makeTask({
      title: "Task due today",
      dueDate: "2026-08-02",
      status: "In-Progress",
    }),
  );

  const tasks = listTasks(
    database,
    {
      archived: false,
      sortBy: "dueDate",
    },
    currentDate,
  );

  const overdueResult = tasks.find(
    (task) => task.id === overdueTask.id,
  );

  const completedResult = tasks.find(
    (task) => task.id === completedTask.id,
  );

  const todayResult = tasks.find(
    (task) => task.id === todayTask.id,
  );

  expect(overdueResult?.isOverdue).toBe(true);
  expect(completedResult?.isOverdue).toBe(false);
  expect(todayResult?.isOverdue).toBe(false);
});

it("sorts tasks by topic, status and due date", () => {
  createTask(
    database,
    makeTask({
      title: "Zebra topic",
      topic: "Zebra",
      status: "Complete",
      dueDate: "2026-08-12",
    }),
  );

  createTask(
    database,
    makeTask({

```

```

    title: "Alpha topic",
    topic: "Alpha",
    status: "Todo",
    dueDate: "2026-08-10",
  }),
);

createTask(
  database,
  makeTask({
    title: "Middle topic",
    topic: "Middle",
    status: "In-Progress",
    dueDate: "2026-08-11",
  }),
);

const byTopic = listTasks(database, {
  sortBy: "topic",
});

const byStatus = listTasks(database, {
  sortBy: "status",
});

const byDueDate = listTasks(database, {
  sortBy: "dueDate",
});

expect(byTopic.map((task) => task.topic)).toEqual([
  "Alpha",
  "Middle",
  "Zebra",
]);

expect(byStatus.map((task) => task.status)).toEqual([
  "Todo",
  "In-Progress",
  "Complete",
]);

expect(
  byDueDate.map((task) => task.dueDate),
).toEqual([
  "2026-08-10",
  "2026-08-11",
  "2026-08-12",
]);
});
});
```

```

These are five real behavioural tests. Every test receives a brand-new in-memory SQLite database through:

```

```typescript
openDatabase(":memory:")
```

```

Nothing depends on `data/tasks.db`, and the database is closed after each test.

## 4. Run the full verification

```

```powershell
npm test
npm run lint
npm run build
git status
```

```

Expected test result:

```

```text
1 test file passed
5 tests passed
```

```

Do not commit yet. Send the complete `npm test`, lint, build and status output so we can correct any issue before locking in the testing commit.

```

PS C:\Code\SDP\coms-3011a-lab-1> npm test
>> npm run lint
>> npm run build
>> git status

> coms-3011a-lab-1@0.1.0 test
> vitest run

RUN v4.1.10 C:/Code/SDP/coms-3011a-lab-1
✓ src/lib/tasks/repository.test.ts (5 tests) 37ms
✓ task repository (5)
 ✓ creates and retrieves a task with normalised input 24ms
 ✓ updates an existing active task 3ms
 ✓ archives a task without deleting it 3ms
 ✓ derives overdue state from the due date and status 2ms
 ✓ sorts tasks by topic, status and due date 3ms

Test Files 1 passed (1)
Tests 5 passed (5)
Start at 20:26:07
Duration 493ms (transform 83ms, setup 0ms, import 151ms, tests 37ms, environment 0ms)

> coms-3011a-lab-1@0.1.0 lint
> eslint

> coms-3011a-lab-1@0.1.0 build
> next build

▲ Next.js 16.2.12 (Turbopack)

 Creating an optimized production build ...
✓ Compiled successfully in 3.3s
✓ Finished TypeScript in 3.1s
✓ Collecting page data using 5 workers in 916ms
✓ Generating static pages using 5 workers (4/4) in 1021ms
✓ Finalizing page optimization in 13ms

Route (app)
└─ f /
 ○ /_not-found

○ (Static) prerendered as static content
f (Dynamic) server-rendered on demand

On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
 (use "git add <file>..." to update what will be committed)
 (use "git restore <file>..." to discard changes in working directory)
 modified: package-lock.json
 modified: package.json

Untracked files:
 (use "git add <file>..." to include in what will be committed)
 src/lib/tasks/repository.test.ts

no changes added to commit (use "git add" and/or "git commit -a")

```

W ☐ The testing slice is complete and passes everything.

You now have:

- **\*\*5 meaningful behavioural tests\*\***
- A fresh `:memory:` SQLite database for every test
- Coverage for create, update, archive, overdue derivation, and all three sorting modes
- No dependency on your personal `data/tasks.db`
- One documented command: ``npm test``
- Passing lint, TypeScript compilation, and production build

That meets the lab's full testing rubric, including the requirement for at least one archive or overdue test and a disposable database. [filecite\turn0file1](#)

## Commit the tests

```
```powershell
git add package.json package-lock.json src/lib/tasks/repository.test.ts

git commit `
-m "test: add task repository behaviour tests" `
-m "Test task creation, editing, archiving, overdue derivation, and sorting using an
isolated in-memory SQLite database." `
-m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"

git push
```
```

Then confirm:

```
```powershell
git status
```
```

The remaining final-session work is documentation, AI transcript preparation, and the clean-clone walkthrough. Separate API routes are not needed.

## User

Message 51

ok Ws whats next

## Assistant

Message 52

Next is the **final documentation**, because it carries 18 marks and must accurately match the repository. The lab specifically requires three Markdown sections: Third-Party Code, Database Design, and Running It. [filecite\turn0file1](#)

Before writing it, send me the output of:

```
```powershell
Get-Content package.json
git log --oneline --reverse
```
```

Then we'll replace the default Next.js README with:

1. **Project overview**
2. **Features**
3. **Third-Party Code**
  - Every directly installed library/package
  - One clear reason for each
4. **Database Design**
  - The `tasks` table and each field
  - `archived\_at` representing archiving
  - Overdue being derived, not stored
  - No relationships because the application currently has one table
5. **Running It**
  - Node.js `v24.13.1`
  - Exact clean-clone install, development, production and test commands
6. **AI Usage**
  - Code generation
  - Code review
  - No inline editing
7. **README document declaration**
  - A separate declaration stating that the README itself was generated/reviewed with ChatGPT-Web[GPT-5.6 Thinking], as required by the writing policy. [filecite\turn0file0](#)

After that, the last steps are:

- Commit the documentation.
- Prepare the AI transcript files.
- Clone the repository into a fresh folder.
- Follow the README alone.
- Perform the complete seven-step marking walkthrough.
- Submit the repository link, documentation and transcripts.

No API routes are needed. The application's frontend and backend are already connected through Server Actions.

this is what my teacher said : For clarity of the submission, please have your documentation in the repo under a folder called 'docs' or something similar. Please submit the transcripts the same way under an 'ai' folder.

That means your submission should be just the repo link. and then this is my terminal : PS

C:\Code\SDP\coms-3011a-lab-1> Get-Content package.json

>> git log --oneline --reverse

```
{
 "name": "coms-3011a-lab-1",
 "version": "0.1.0",
 "private": true,
 "scripts": {
 "dev": "next dev",
 "build": "next build",
 "start": "next start",
 "lint": "eslint",
 "test": "vitest run"
 },
 "dependencies": {
 "better-sqlite3": "^13.0.2",
 "next": "16.2.12",
 "react": "19.2.4",
 "react-dom": "19.2.4"
 },
 "devDependencies": {
 "@tailwindcss/postcss": "^4",
 "@types/better-sqlite3": "^7.6.13",
 "@types/node": "^20",
 "@types/react": "^19",
 "@types/react-dom": "^19",
 "eslint": "^9",
 "eslint-config-next": "16.2.12",
 "tailwindcss": "^4",
 "typescript": "^5",
 "vitest": "^4.1.10"
 }
}
ccd6640 Initial commit
eeb4da9 chore: scaffold Next.js application
a60ae21 feat: add SQLite database foundation
c506f5b feat: add task domain and SQLite repository
aceaeb6 feat: add task creation and active task list
0693679 feat: add task editing archiving and sorting
0b130b1 (HEAD -> main, origin/main, origin/HEAD) test: add task repository behaviour tests
```

Perfect. Your lecturer's clarification means the **repository** itself becomes the complete submission, so we should use this structure:

```
``text
COMS-3011A-Lab-1/
├── README.md
├── docs/
│ └── documentation.md
├── ai/
│ ├── README.md
│ └── chatgpt-transcript.pdf
├── database/
├── src/
└── ...
...

```

## Important README decision

Even though the detailed documentation will be under `docs/`, the top-level `README.md` must still contain the exact clean-clone installation and startup commands. The marking walkthrough begins by testing whether the application can be installed and started by following the README alone. [filecite1](#)

So:

### `README.md`

Keep it concise but include:

- Project description

- Feature summary
- Exact quick-start commands
- Exact test command
- Link to `docs/documentation.md`
- Link to the AI transcript folder
- Repository AI usage declaration
- README writing declaration

### `docs/documentation.md`

Include the three required sections in full:

1. **Third-Party Code**
2. **Database Design**
3. **Running It**

We can also include a testing section and design decisions.

### `ai/`

The safest transcript format is an **unedited PDF export of this conversation**:

```
```text
ai/chatgpt-transcript.pdf
```
```

Because all three sessions occurred in this same conversation, one complete PDF should cover:

- Planning
- Code generation
- Debugging and verification
- Your constraints and corrections

The transcript already contains a clear example of you rejecting an unsuitable suggestion: you rejected creating a separate trivial commit solely for the README AI-review change and directed that it be included in the next meaningful commit. That is useful for the AI-usage rubric. [filecite\[turn0file1\]](#)

`ai/README.md` should identify:

- Tool: ChatGPT Web
- Model: GPT-5.6 Thinking
- Uses: planning, code generation, debugging, review and documentation
- The transcript filename
- A statement that the transcript is unedited

The raw transcript itself should not be rewritten or cleaned up.

## Commit history status

You now have **seven commits**:

```
```text
Initial commit
Scaffold
Database foundation
Task repository
Create and list
Edit, archive and sorting
Tests
```
```

That exceeds the six-commit minimum. The commits are coherent, descriptive, and visibly spread across multiple sessions, so the history is currently positioned for the full commit-history mark. [filecite\[turn0file1\]](#)

## Package documentation scope

In `Third-Party Code`, we should document every **direct dependency and development dependency** shown in `package.json`, each with a one-line reason:

- `next`
- `react`
- `react-dom`
- `better-sqlite3`
- `vitest`
- `typescrypt`
- `tailwindcss`
- `@tailwindcss/postcss`
- `eslint`

```
- `eslint-config-next`
- `@types/node`
- `@types/react`
- `@types/react-dom`
- `@types/better-sqlite3`
```

We do not need to document every transitive package in `package-lock.json`.

## Environment to document

```
```text  
Node.js: v24.13.1  
npm: 11.8.0  
```
```

Create the folders now:

```
```powershell  
New-Item -ItemType Directory -Force docs  
New-Item -ItemType Directory -Force ai  
```
```

Do not commit the empty folders—Git does not track empty directories anyway. The next step is replacing the default README and creating `docs/documentation.md` and `ai/README.md` as one meaningful documentation commit.

## User

Message 55

alright whats next

## Assistant

Message 56

Next, create the three documentation files. These must include the exact required headings and AI declarations. The lab requires **Third-Party Code**, **Database Design**, and **Running It**, while the AI policy requires every document to declare AI usage or non-usage.

[filecite\turn0file1](#) [filecite\turn0file0](#)

## 1. Replace `README.md`

Replace the current generated README with:

```
```markdown  
# COMS3011A Lab 1 — Local Task Manager
```

A local-first todo application built with Next.js and SQLite.

The application runs locally on a single user's computer. It does not require user accounts or web deployment.

Features

- Create tasks with a title, description, due date, topic and status.
- Edit active tasks.
- Archive tasks without deleting them.
- View active and archived tasks separately.
- Sort tasks by topic, status or due date.
- Use one of three fixed statuses:
 - Todo
 - In-Progress
 - Complete
- Visually identify overdue incomplete tasks.
- Persist task information between application restarts.

Quick Start

Requirements

This project was developed and tested using:

```
- Node.js `v24.13.1`  
- npm `11.8.0`
```

Install

From a clean clone of the repository:

```
```bash  
npm ci
```
```

Run in Development

```
```bash
npm run dev
```
```

Open the application at:

```
```text
http://localhost:3000
```
```

The local SQLite database is created automatically when the application first runs.

Run the Tests

```
```bash
npm test
```
```

Run a Production Build

```
```bash
npm run build
npm start
```
```

Documentation

Detailed project documentation is available in:

- [Project Documentation](docs/documentation.md)
- [AI Usage Records](ai/README.md)

AI Usage

This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

This repository does not use AI in-line editing tools.

This repository makes use of AI code-review using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

2. Create `docs/documentation.md`

```
```markdown
Project Documentation
```
```

Third-Party Code

The following direct dependencies and development dependencies are installed by this project.

| Package | Purpose |
|----------------------|---|
| next | Provides the application framework, App Router, Server Components and Server Actions used to build the local web application. |
| react | Provides the component-based user interface system used by Next.js. |
| react-dom | Renders React components in the browser and supports React's server-rendering integration. |
| better-sqlite3 | Provides synchronous access to the local SQLite database from the server-side application code. |
| vitest | Runs the automated behavioural tests for the task repository. |
| typescript | Provides static type checking for the application and database code. |
| tailwindcss | Provides utility classes used to style the user interface. |
| @tailwindcss/postcss | Connects Tailwind CSS to the project's PostCSS build process. |
| eslint | Performs static analysis and identifies code-quality problems. |
| eslint-config-next | Supplies the recommended ESLint configuration for a Next.js application. |
| @types/node | Provides TypeScript definitions for Node.js APIs such as the filesystem and path modules. |
| @types/react | Provides TypeScript definitions for React. |
| @types/react-dom | Provides TypeScript definitions for React DOM. |

```
| `@types/better-sqlite3` | Provides TypeScript definitions for the `better-sqlite3` package.
|
```

The project does not copy third-party source code directly into the repository. Installed packages are recorded in `package.json` and locked to resolved versions through `package-lock.json`.

Database Design

The application uses SQLite and contains one table named `tasks`.

Tasks Table

```
| Column | Type | Description |
|---|---|---|
| `id` | `INTEGER` | Primary key that uniquely identifies each task. It automatically
increments when a task is created. |
| `title` | `TEXT` | Required title of the task. A database constraint prevents an empty or
whitespace-only value. |
| `description` | `TEXT` | Additional task details. It defaults to an empty string. |
| `due_date` | `TEXT` | Required due date stored in `YYYY-MM-DD` format. This format allows
dates to be sorted chronologically as text. |
| `topic` | `TEXT` | Required topic used to categorise and sort tasks. |
| `status` | `TEXT` | Current task status. A database constraint restricts it to `Todo`,
`In-Progress` or `Complete`. |
| `archived_at` | `TEXT` or `NULL` | Stores the date and time at which a task was archived. A
`NULL` value means that the task is active. |
| `created_at` | `TEXT` | Timestamp automatically recorded when the task is created. |
| `updated_at` | `TEXT` | Timestamp recorded when the task is created and updated whenever the
task is edited or archived. |
```

Relationships

There are no relationships between tables because the application requires only one table.

Topics are stored directly on each task because topics are simple task attributes and are not independently created or managed by the user.

Statuses are also stored directly on each task. They are restricted to the three fixed values required by the brief and are not user-customisable.

Archiving

Archived tasks remain in the `tasks` table.

The application sets `archived\_at` to a timestamp when a task is archived. Active tasks have an `archived\_at` value of `NULL`.

Tasks are never deleted or copied into a separate archive table. This allows archived tasks to remain viewable while preserving their original identity and information.

Overdue Calculation

Overdue is not stored as a database column or status.

It is derived when tasks are read from the database. A task is overdue when:

1. Its due date is earlier than the current local date.
2. Its status is not `Complete`.

A task due on the current date is not overdue. A completed task is not displayed as overdue even when its due date is in the past.

Indexes

The schema includes indexes on:

```
- `topic`
- `status`
- `due_date`
- `archived_at`
```

These support the required task sorting and active/archive filtering operations.

Database Location

The development database is stored at:

```
```text
data/tasks.db
```

...

The `data` directory and generated SQLite files are excluded from Git. A new database is created automatically from `database/schema.sql` when the application is run from a clean clone.

Tests do not use the development database. Each test uses a new in-memory SQLite database that is discarded after the test finishes.

## Running It

### Required Software

The project was developed and verified using:

```
```text
Node.js v24.13.1
npm 11.8.0
```
```

### Starting From a Clean Clone

Clone the repository:

```
```bash
git clone https://github.com/CalebP987/COMS-3011A-Lab-1.git
```
```

Enter the repository directory:

```
```bash
cd COMS-3011A-Lab-1
```
```

Install the exact dependencies recorded in `package-lock.json`:

```
```bash
npm ci
```
```

Start the development server:

```
```bash
npm run dev
```
```

Open the following address in a browser:

```
```text
http://localhost:3000
```
```

The database directory, database file and task table are created automatically when the application first loads.

### Running Automated Tests

Run all tests using the single documented command:

```
```bash
npm test
```
```

The test suite covers:

- Task creation and retrieval.
- Task editing.
- Task archiving without deletion.
- Overdue-state calculation.
- Sorting by topic, status and due date.

Each test uses an isolated in-memory SQLite database.

### Linting

```
```bash
npm run lint
```
```

### Production Build

Create an optimised production build:

```
```bash
npm run build
```
```

Start the production server:

```
```bash
npm start
```
```

Open:

```
```text
http://localhost:3000
```
```

### ### Available npm Commands

| Command         | Purpose                                       |
|-----------------|-----------------------------------------------|
| `npm run dev`   | Starts the Next.js development server.        |
| `npm run build` | Creates an optimised production build.        |
| `npm start`     | Runs the completed production build.          |
| `npm run lint`  | Runs ESLint.                                  |
| `npm test`      | Runs all behavioural tests once using Vitest. |

### ## AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

### ## 3. Create `ai/README.md`

```
```markdown
# AI Usage Records
```
```

### ## Tools Used

| Tool        | Model            | Uses                                                                                                                      |
|-------------|------------------|---------------------------------------------------------------------------------------------------------------------------|
| ChatGPT Web | GPT-5.6 Thinking | Planning, architecture discussion, code generation, debugging, code review, test generation and documentation assistance. |

### ## Transcript

The complete ChatGPT conversation used during this assessment is included in this directory as:

```
```text
chatgpt-transcript.pdf
```
```

The transcript covers:

- Review of the COMS3011A AI policy and Lab 1 brief.
- Project and database planning.
- Database schema and repository development.
- Frontend and Server Action development.
- Debugging and manual verification.
- Automated test development.
- Documentation preparation.
- Final submission verification.

The transcript is submitted unedited so that the original prompts, generated outputs, constraints, corrections and decisions remain visible.

### ## AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

After creating those files, run:

```
```powershell
npm test
```
```

```
npm run lint
npm run build
git status
````
```

Do not commit yet. The next step will be exporting this complete conversation to `ai/chatgpt-transcript.pdf`, checking every document against the policy, and then making the documentation commit.

User

Message 57

the documentation links dont work and after that you said alot of stuff, just give me the commands to do dont say create .md file give me the stuff to put into terminal or file to make be specific now

Assistant

Message 58

Run these **from the repository root**:

```
``powershell
PS C:\Code\SDP\coms-3011a-lab-1>
```

1. Create the folders

```
``powershell
New-Item -ItemType Directory -Force .\docs
New-Item -ItemType Directory -Force .\ai
````
```

**## 2. Replace `README.md`**

Copy and paste this entire command into PowerShell:

```
``powershell
@'
COMS3011A Lab 1 — Local Task Manager
```

A local-first todo application built with Next.js and SQLite.

The application runs locally for a single user. It does not require user accounts or deployment.

**## Features**

- Create tasks with a title, description, due date, topic and status.
- Edit active tasks.
- Archive tasks without deleting them.
- View active and archived tasks separately.
- Sort tasks by topic, status or due date.
- Use the fixed statuses `Todo`, `In-Progress` and `Complete`.
- Visually identify overdue incomplete tasks.
- Persist task information after the application is restarted.

**## Quick Start**

**### Requirements**

This project was developed and tested using:

- Node.js `v24.13.1`
- npm `11.8.0`

**### Install**

From the repository directory:

```
~~~bash
npm ci
~~~
```

**### Run in Development**

```
~~~bash
npm run dev
~~~
```

Open:

```
~~~text
http://localhost:3000
~~~
```

The SQLite database is created automatically when the application first runs.

### ### Run Tests

```
~~~bash
npm test
~~~
```

### ### Run a Production Build

```
~~~bash
npm run build
npm start
~~~
```

Open:

```
~~~text
http://localhost:3000
~~~
```

### ## Documentation

- [Project documentation](docs/documentation.md)
- [AI usage records](ai/README.md)

### ## AI Usage

This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

This repository does not use AI in-line editing tools.

This repository makes use of AI code-review using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

### ## AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].  
'@ | Set-Content -Encoding UTF8 .\README.md  
``

### ## 3. Create `docs/documentation.md`

```
```powershell
@'
# Project Documentation
```

Third-Party Code

The following direct dependencies and development dependencies are installed by this project.

| Package | Reason for Use |
|-----------------------|---|
| next | Provides the application framework, App Router, Server Components and Server Actions. |
| react | Provides the component-based user interface system used by Next.js. |
| react-dom | Renders React components and supports React server rendering. |
| better-sqlite3 | Provides synchronous server-side access to the local SQLite database. |
| vitest | Runs the automated behavioural tests. |
| typescript | Provides static type checking for the application and database code. |
| tailwindcss | Provides utility classes used to style the user interface. |
| @tailwindcss/postcss | Integrates Tailwind CSS with the PostCSS build process. |
| eslint | Performs static analysis and identifies code-quality problems. |
| eslint-config-next | Supplies the recommended ESLint rules for Next.js. |
| @types/node | Provides TypeScript definitions for Node.js APIs. |
| @types/react | Provides TypeScript definitions for React. |
| @types/react-dom | Provides TypeScript definitions for React DOM. |
| @types/better-sqlite3 | Provides TypeScript definitions for better-sqlite3. |

Installed packages are recorded in `package.json`. Exact resolved versions are recorded in `package-lock.json`.

Database Design

The application uses SQLite and contains one table named `tasks`.

Tasks Table

| Column | Type | Description |
|---------------|------------------|--|
| `id` | `INTEGER` | Auto-incrementing primary key that uniquely identifies a task. |
| `title` | `TEXT` | Required task title. Empty or whitespace-only titles are rejected. |
| `description` | `TEXT` | Additional task details. Defaults to an empty string. |
| `due_date` | `TEXT` | Required date stored in `YYYY-MM-DD` format. |
| `topic` | `TEXT` | Required task topic used for display and sorting. |
| `status` | `TEXT` | Restricted to `Todo`, `In-Progress` or `Complete`. |
| `archived_at` | `TEXT` or `NULL` | Archive timestamp. `NULL` means that the task is active. |
| `created_at` | `TEXT` | Timestamp recorded when the task is created. |
| `updated_at` | `TEXT` | Timestamp updated when the task is edited or archived. |

Relationships

There are no relationships between tables because the application requires only one table.

Topics are stored directly on tasks because topics are simple attributes and are not independently managed.

Statuses are also stored directly on tasks. A database constraint restricts them to the three fixed values required by the brief.

Archiving

Archived tasks remain in the `tasks` table.

When a task is archived, the application sets `archived\_at` to a timestamp. Active tasks have an `archived\_at` value of `NULL`.

Tasks are never deleted or moved into a separate archive table.

Overdue Calculation

Overdue is not stored as a database column or task status.

It is calculated when a task is read. A task is overdue when:

1. Its due date is earlier than the current local date.
2. Its status is not `Complete`.

A task due on the current date is not overdue. A completed task is not shown as overdue, even when its due date has passed.

Indexes

The schema contains indexes on:

- `topic`
- `status`
- `due\_date`
- `archived\_at`

These support sorting and active/archive filtering.

Database Location

The application database is stored at:

```
~~~text
data/tasks.db
~~~
```

The `data` directory and generated SQLite files are excluded from Git.

A new database is created automatically using `database/schema.sql` when the application runs from a clean clone.

Tests do not use the development database. Each test uses a new in-memory SQLite database that is discarded after the test.

Running It

Required Software

The project was developed and verified using:

```
~~~text
Node.js v24.13.1
npm 11.8.0
~~~
```

Starting From a Clean Clone

Clone the repository:

```
~~~bash
git clone https://github.com/CalebP987/COMS-3011A-Lab-1.git
~~~
```

Enter the repository:

```
~~~bash
cd COMS-3011A-Lab-1
~~~
```

Install the exact dependencies from `package-lock.json`:

```
~~~bash
npm ci
~~~
```

Start the development server:

```
~~~bash
npm run dev
~~~
```

Open:

```
~~~text
http://localhost:3000
~~~
```

The database directory, database file and task table are created automatically when the application first loads.

Running Automated Tests

Run all tests with:

```
~~~bash
npm test
~~~
```

The test suite covers:

- Task creation and retrieval.
- Task editing.
- Task archiving without deletion.
- Overdue-state calculation.
- Sorting by topic, status and due date.

Each test uses an isolated in-memory SQLite database.

Linting

```
~~~bash
npm run lint
~~~
```

Production Build

Create an optimised production build:

```
~~~bash
npm run build
~~~
```

Start the production server:

```
~~~bash
npm start
~~~
```

Open:

```
~~~text
http://localhost:3000
~~~
```

Available npm Commands

```
| Command | Purpose |
|---|---|
| `npm run dev` | Starts the Next.js development server. |
| `npm run build` | Creates an optimised production build. |
| `npm start` | Runs the production build. |
| `npm run lint` | Runs ESLint. |
| `npm test` | Runs all tests once using Vitest. |
```

AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

```
'@ | Set-Content -Encoding UTF8 .\docs\documentation.md
`
```

4. Create `ai/README.md`

```
` powershell
@'
# AI Usage Records
```

Tools Used

```
| Tool | Model | Uses |
|---|---|---|
| ChatGPT Web | GPT-5.6 Thinking | Planning, architecture discussion, code generation,
debugging, code review, test generation and documentation assistance. |
```

Transcript

The complete ChatGPT conversation used during this assessment will be stored in this directory as:

```
~~~text
chatgpt-transcript.pdf
~~~
```

The transcript covers:

- Review of the COMS3011A AI policy and Lab 1 brief.
- Project and database planning.
- Database schema and repository development.
- Frontend and Server Action development.
- Debugging and manual verification.
- Automated test development.
- Documentation preparation.
- Final submission verification.

The transcript is submitted unedited so that the original prompts, generated outputs, constraints, corrections and decisions remain visible.

AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

```
'@ | Set-Content -Encoding UTF8 .\ai\README.md
`
```

5. Confirm the files and links exist

```
` powershell
Get-ChildItem .\docs
Get-ChildItem .\ai
git status
`
```

You should see:

```
` text
docs\documentation.md
ai\README.md
`
```

Then open the README preview in VS Code. The links will now work because the target files actually exist:

```
```powershell
code .\README.md
```
```

Do not commit yet. The transcript PDF still needs to be placed at:

```
```text
ai\chatgpt-transcript.pdf
```
```

User

Message 59

ok ill do that last and commit at the end after i do that whats next

Assistant

Message 60

After you add `ai/chatgpt-transcript.pdf`, do this:

1. Verify everything is present

From the repo root:

```
```powershell
Get-ChildItem .\docs
Get-ChildItem .\ai
git status
```
```

You should see:

```
```text
docs\documentation.md
ai\README.md
ai\chatgpt-transcript.pdf
```
```

2. Run the final checks

```
```powershell
npm test
npm run lint
npm run build
```
```

All three must pass.

3. Commit and push the documentation

```
```powershell
git add README.md docs ai

git commit `
 -m "docs: add project documentation and AI records" `
 -m "Document dependencies, database design, clean-clone instructions, and include the
 complete AI usage transcript."
```
```

```
git push
```
```

The Markdown files already contain their required AI declarations, so an `Assisted-by` footer is not required for this documentation-only commit. The policy requires document declarations for AI-assisted writing and commit attribution specifically for AI-generated code.

[filecite](#)[turn0file0](#)

Check:

```
```powershell
git status
git log --oneline --decorate -10
```
```

The working tree should be clean.

## 4. Test from a completely clean clone

Leave your existing repository untouched:

```
```powershell
cd C:\Code\SDP
```

```
git clone https://github.com/CalebP987/COMS-3011A-Lab-1.git coms-3011a-lab-1-clean
```

```
cd .\coms-3011a-lab-1-clean
npm ci
npm test
npm run lint
npm run build
npm run dev
```
```

Open:

```
```text
http://localhost:3000
```
```

### 5. Perform the lecturer's walkthrough

In the clean clone, confirm these in order:

1. The app installs and starts using the README alone.
2. Create a task with title, description, due date and topic.
3. Edit it and reload the page.
4. Archive it and confirm it remains in the archived view.
5. Test sorting by topic, status and due date.
6. Create a past incomplete task and confirm it is visibly overdue without "Overdue" appearing as a selectable status.
7. Stop the server, restart it with `npm run dev`, and confirm the data persists.

That is the exact functional walkthrough used for 28 marks. [filecite](#)[turn0file1](#)

Once the clean-clone walkthrough passes, the repository link is ready to submit.

**User**

Message 61

im trying to convert this chat to pdf but its giving issues its only getting part of our chat, I want this full chat pls do it yourself and send me a pdf of everything weve done in this project which is just this "chat", no need to redo any github things just send me the pdf nothing more